Amplifier Simulation User Manual

Amplifier Simulation models Pump absorption, Seed amplification, energy-level populations, and thermal effects in solid-state laser amplifiers. This manual explains how to set up a calculation, choose the models, and read the results.

For installation and launch, start with the README. In the interface, choose **English** from the language selector. The question-mark icons beside the controls explain the settings and options.

Calculations use SI units internally. The interface shows units such as mm, µm, nm, fs, and nJ where indicated. The source code, defaults, and validators define which settings the program supports.

The fast model is called **Commercial App**. Its configuration identifier is `commercial_app_fast`; related keys and module names use `commercial_app`.

The source package includes `configs/example.json`, with its pulse inputs embedded. Other material libraries, study definitions, experimental inputs, research configurations, developer tests, and historical results must be supplied separately. The research validation and literature notes are also separate from this package.

Source release: 18 October 2026. Manual updated: 30 October 2026. Section 24.5 records the earlier source review and its scope.

Contents

1. Program scope and basic concepts
2. Installation, environment, and startup
3. Recommended modeling workflow
4. Interface overview
5. Importing JSON and material libraries
6. Run modes, steady state, and solvers
7. Crystal, materials, and energy levels
8. Pump module
9. Seed module
10. Multipass propagation and the optical bench
11. Heat, stress, dispersion, and nonlinearity
12. Numerical grid
13. Parameter sweeps
14. Running calculations and viewing results
15. Steady state, cycles, and time samples per cycle
16. Command line, batch processing, and reproducible runs
17. JSON configuration reference
18. Module coupling, conservation, and avoiding double counting
19. Performance, memory, and Apple Silicon
20. Frequently asked questions and troubleshooting
21. Choosing models by research question
22. Output files and fields
23. Source module index
24. Current limitations, verification checklist, and glossary

The final part of Section 24 also covers the optional Commercial App radial grid, the Spectral Time radial interface and binning, initial-energy tail binning, final-output diagnostic timing, and joint thermal-optical iteration.

1. Purpose and Basic Concepts

1.1 What the Program Models

The program models three-dimensional solid-state laser amplifiers along the crystal's main propagation direction. It tracks:

- Pump fields and intensity, $I_p(x, y, z, t, \lambda)$.
- Seed fluence or complex field, $F_s(x, y, z, t, \lambda)$ / $E_s(x, y, z, t)$.
- Populations of each energy level, $N_i(x, y, z, t)$.
- Local net gain: stimulated emission minus reabsorption, followed by passive loss:

$$g(\lambda) = \sigma_e(\lambda)N_u - \sigma_a(\lambda)N_l - \alpha_{passive}.$$

- Pump absorption, active Seed extraction, ASE, heat sources, temperature, stress approximations, and refractive-index changes.
- Multiple passes, external lenses/mirrors/interfaces, and diffraction inside the crystal.

1.2 Three Quantities That Are Easily Confused

Inversion density is usually displayed as

$$\Delta N = N_u - N_l,$$

whereas the actual gain uses the cross-section-weighted quantity $\sigma_e N_u - \sigma_a N_l$. Positive inversion therefore does not necessarily imply positive gain at every wavelength.

Pulse energy is the energy of one Seed pulse. **Average power** is calculated from

$$P_{avg} = E_{pulse} f_{rep}.$$

A repetition rate of zero specifies a single shot. The summary may then report zero power from this product; this does not imply zero pulse energy. That value does not describe the average power of repetitive operation.

Absorbed Pump power may be a steady-state value, a Pump-cycle average, or a time average over the single-shot preparation window, depending on how populations are evolved. Its units are power, rather than joules per cycle. Multiply the average absorbed power by the sampling-window duration to obtain absorbed energy. Volumetric heat sources have units of W/m³.

1.3 Scope of the Physical Model

The models use rate equations, slowly varying envelopes, and paraxial propagation. They account for population depletion, gain saturation, coherent spectral phase, dispersion, and Kerr effects. They do not solve optical polarization, finite dephasing, or Rabi oscillations. Pulses of tens of femtoseconds can be modeled where the envelope approximation remains valid; few-cycle carrier dynamics and strongly coherent transitions require a separate Maxwell-Bloch/UPPE backend.

2. Installation, Environment, and Launch

2.1 Recommended Environment

The project requires Python 3.10 or later. Python 3.10, 3.11, or 3.12 is recommended for this release. Extract the ZIP, open a terminal in the `AmplifierSimulation` folder containing `app.py`, and create a new environment.

macOS / Linux:

```
python3 -m venv .venv
.venv/bin/python -m pip install ".[ui,performance]"
.venv/bin/python -m streamlit run app.py
```

Windows, using PowerShell or Command Prompt:

```
py -m venv .venv
.venv\Scripts\python.exe -m pip install ".[ui,performance]"
.venv\Scripts\python.exe -m streamlit run app.py
```

The browser should open automatically. If it does not, visit `http://localhost:8501`. Select **English** in the sidebar if needed, open **Configuration File**, and upload `configs/example.json`. The example's values fill the form automatically.

To avoid typing the Python path for each command, activate the environment: use `source .venv/bin/activate` on macOS/Linux, `.venv\Scripts\activate.bat` in Windows Command Prompt, or `.venv\Scripts\Activate.ps1` in PowerShell if your execution policy permits it. Activation applies only to the current terminal session. The commands above use the full Python path and work without activation.

If you use conda, run `conda activate <environment-name>` and use that interpreter for both installation and launch. The ZIP does not require or include a pre-existing local environment or an ARM-specific environment folder. Later `amplifier-sim` commands assume an activated environment. Without activation, substitute `.venv/bin/python -m amplifier_sim` on macOS/Linux or `.venv\Scripts\python.exe -m amplifier_sim` on Windows for `amplifier-sim`.

2.2 Installation Options

Run installation commands from the folder containing `pyproject.toml`, using your environment's Python interpreter:

- Core package: `python -m pip install .`; the only required runtime library is NumPy.
- Web interface: `python -m pip install ".[ui]"`; adds Streamlit, Plotly, and Pandas.
- Performance features: `python -m pip install ".[performance]"`; adds SciPy for multithreaded FFTs, sparse mapping, and the thermal CG solver, plus Numba for a fused pixel-parallel kernel for single-transition coherent transient gain. Short tasks retain the NumPy path automatically to avoid JIT startup overhead.
- Recommended: `python -m pip install ".[ui,performance]"`.
- FROG/HDF5 import: `python -m pip install ".[frog]"`; adds h5py.
- Development tests require `python -m pip install pytest` separately. Quick PNG export requires Pillow: `python -m pip install Pillow`.

For development, use `-e`, as in `python -m pip install -e ".[ui,performance]"`, so source edits take effect without reinstalling. This distribution does not include the separate development tests or historical research fixtures.

2.3 Checking the Installation

With the environment activated:

```
amplifier-sim models
amplifier-sim validate configs/example.json
```

The `validate` command reads the JSON, merges defaults, checks configuration requirements, and prints the resulting configuration. A successful check confirms that the configuration is accepted; it does not validate propagation results or establish agreement with experiment.

For a development checkout containing the test suite, install pytest and run `python -m pytest -q tests`. The full suite also checks historical research configurations and material data that are not included in this ZIP.

If you see `No module named streamlit`, the current `python` usually differs from the interpreter in which Streamlit was installed. Check:

```
python -c "import sys; print(sys.executable)"
python -m pip show streamlit
```

On macOS/Linux, `which python` also shows the interpreter selected by the shell; on Windows, use `where python` in Command Prompt. Use the same environment interpreter for both installation and launch.

2.4 Load, Edit, and Save the Included Example

After launching, select **Amplifier Simulation** and **Configuration Entry → Form**. Open **Configuration File**, upload `configs/example.json`, and wait for the form to refill. The example already contains its material parameters and pulse-input data; you do not need to upload a material library or the original spectrum files.

The following controls change the quantities commonly used in a first comparison. Enter distances and lengths in metres, as shown in their labels, and confirm each numeric entry with Enter or by leaving the field.

Quantity	Where to edit it	Included example
Pump power	**② Beams → Pump ・ Power, Spatial, and Spectral → Pump Power (W)**	12.765 W
Gain-crystal length	**③ Medium → Crystal, Material, and Energy Levels → Crystal Length (m)**	0.015 m = 15 mm; this is separate from the preceding ZnSe thickness
Pump lens-to-crystal distance	Pump panel, section **D. Independent Pump Path and Radius Probe**: edit the `gain_medium` row named `CrZnS_pump_path`	0.150 m = 150 mm
Seed lens-to-crystal distance	Seed panel, **F. Independent Seed Path and Radius Probe**: edit the `gain_medium` row named `CrZnS_seed_path`	0.200 m = 200 mm

In either optical-path table, edit **Distance from Previous Element (m)** in the GAIN row to change the lens-to-crystal distance. The table writes the edit back immediately. Alternatively, choose the GAIN row in **Select Element to Edit**, change **Previous Element → GAIN Input Face Distance (m)**, and click **Apply Selected Parameters Below**. Edits in that lower panel are not applied until you click the button. The LASER-to-first-element field and the first lens row control the source-to-lens distance instead; both are 0.005 m in this example. Pump and Seed have separate paths.

To save the edited configuration before running, switch **Configuration Entry → Full JSON**, copy the complete JSON from the text area, and save it as a `.json` file, for example `my_setup.json`. Click **Apply to Form** to return to the form with those values. Then click **Run Current Configuration**. The button is disabled while another job is active or the configuration has unresolved errors; the interface identifies the reason. In Full JSON mode, **Run JSON** runs the text directly instead.

After a run, **Diagnostics and Export → Download this configuration JSON** saves the configuration used for that result. Later form edits do not alter the saved result; run again to calculate the new settings. Keep a separate JSON for each comparison.

3. Recommended Modeling Workflow

Build an interpretable model in stages:

1. **Align in air:** temporarily remove the gain crystal in the optical-path preview on the right. Adjust only the Pump/Seed sources, distances, and lenses until the caustics in air agree with measurements.
2. **Insert the crystal:** restore its refractive index and interfaces. Check the internal beam size, focal shift, and computational-window coverage.
3. **Test fixed gain:** specify `gain_override_m_inv` first to check the Seed path, multipass losses, and output definitions.
4. **Use Pump-derived population gain:** remove the fixed-gain override and enter ion density, cross sections, and lifetimes. Check Pump absorption and the populations after pumping.
5. **Select the steady-state treatment:** choose Pump-only, average-photon-flux steady state, or a strict pulse map according to repetition rate and single-pulse depletion.
6. **Add thermal effects:** start with a one-way thermal calculation, then enable the thermal-optical fixed point if needed.
7. **Add spectral/time resolution:** use `spectral_time` to examine gain narrowing, then use `coherent_cpa` for phase and compression.
8. **Add rollover mechanisms:** once material data are available, enable temperature-dependent cross sections, ASE, strong-field loss, and Seed heating one at a time.
9. **Check convergence:** increase Nx/Ny/Nz; separately check the FFT windows and sampling for Seed and Pump along each axis, then increase temporal/spectral samples until the main results stop changing appreciably. For automatic axis-by-axis comparison, use `amplifier-sim convergence configs/example.json --factors 1 1.5 2 -o convergence.json`, or replace the configuration with your saved setup.
10. **Run parameter sweeps:** scan Pump power or beam size only after the single-point configuration is credible.

Save the JSON configuration after adding each module so that output changes can be traced to a specific physical setting.

For quantitative calculations, set `simulation.validation_mode` to `strict`. Any blocking failure marks the result invalid. An invalid strict CLI run returns exit code 2 and writes only `summary.invalid.json`; it does not commit new quantitative field files. The interface still offers a **complete diagnostic archive**, marked non-quantitative, containing the run configuration, 3D fields, optical planes, and warnings needed to reproduce the failure. This diagnostic download is separate from valid-result export. A failed run leaves existing quantitative output directories unchanged, so do not pair an older `fields.npz` with the new diagnostics.

Validity reports separate `blocking` failures from `warning` diagnostics. Blocking checks cover accounting, conservation, convergence, applicability, and actual periodic FFT boundaries. A warning may concern only a recorded plane, beam size, or M^2; it does not necessarily invalidate power metrics. Sweeps record four states-valid, valid-with-warnings, completed-but-invalid, and execution-failed-and retain each metric's validity separately.

Completed invalid operating points retain their raw diagnostic values. Valid 1D trends break at missing values, invalid metrics, and backend changes; invalid runs can optionally be displayed as red × marks. In live 2D sweeps, **Include raw metrics from validation-invalid grid points** adds available raw values to the heatmap and slice lines. It defaults to on in exploratory mode and off in strict mode; it does not change run validity. Disable it for quantitative comparisons. Archived-result analysis organizes trends by valid runs and valid metrics, displaying different actual Seed backends separately.

Validation-invalid interface views, CLI outputs, and archives display `NON-QUANTITATIVE — VALIDATION FAILED`. Saved diagnostics remain unsuitable for quantitative interpretation. With `require_experimentally_calibrated_material=true`, the material snapshot must include a sample identifier and calibration-data SHA-256, and `calibrated_parameters` must cover every `required_parameters` entry.

Material tables use `range_policy=record` by default and record clamping at the lower/upper boundary. To stop before calculation when values are out of range, set `material.spectroscopic_response.range_policy="error"`.

By default, strict validation requires both positive and negative Seed energy residuals to be closed by named terms. To use the older one-sided check that only rejects artificial energy generation, explicitly set `simulation.strict_validation.require_named_external_losses=false`; `closed_seed_energy_ledger` is then marked not applicable. Combined ABCD/FFT tilt and the Nyquist occupancy of a tilted FFT carrier also enter the applicability checks. Quantitative strict ASE runs require an explicit `ase.radiative_lifetime_s`; the total upper-state lifetime, which may include nonradiative quenching, must not silently be treated as the radiative lifetime.

Spatial-boundary checks block a result only for internal propagation paths that use periodic transverse FFTs. A separate blocking check measures captured power when a hybrid Pump/Seed path converts an analytical ABCD field to the crystal FFT grid. For a source display or ordinary recorded plane, an undersized window instead prompts you to enlarge the diagnostic window before interpreting beam size/M². Omission of infinite-plane Gaussian tails from a display does not by itself show physical clipping or FFT wraparound.

---

4. Overall Interface Layout

Inputs and model choices appear on the left. The current solver requirements, optical-path preview, results, and diagnostics appear on the right. The left side uses five workspaces sharing one configuration:

1. ① **Model:** solver family, gain source, periodic steady state, multipass organization, and thermal-optical coupling.
2. ② **Beams:** Pump/Seed sources, temporal and spectral definitions, spatial modes, polarization, and separate optical paths.
3. ③ **Medium:** crystal, material, energy levels, heating, cooling, stress, ASE, spectroscopic feedback, and strong-field nonlinearities.
4. ④ **Numerics:** physical/optical grids, sampling, computational budgets, and validity checks.
5. ⑤ **Result Recording:** recording level, diagnostic timing, longitudinal samples, and optional field/plane output.

Interface Level → Guided keeps the common controls visible and collapses advanced solver overrides. **Expert** exposes the detailed overrides. Both edit the same configuration; changing the interface level does not itself enable or disable physical models.

Use the question-mark icon beside a control to read its help text. For an explanation of the current calculation as a whole, click **Generate / Refresh Complete Model Description** in the configuration-check area or the equivalent button in the sidebar, or **Generate Complete Physics Description** below the optical-path preview. The generated explanation shows the effective Pump, population, gain, thermal, and Seed chain, including disabled or locked features. It follows the current form and can be downloaded as Markdown. Generating it does not start a numerical calculation.

Within Pump, the form is divided into A: Power and Active Absorption; B: Spatial Source and Propagation; C: Temporal Waveform and Spectrum; and D: Separate Optical Path and Radius Probes. Seed is divided into A: Pulse; B: Gain Transport; C: Polarization; D: Spatial; E: Complex Spectrum/CPA; and F: Separate Optical Path.

The multipass area contains one set of templates, crystal encounters, and inter-pass paths. The legacy reverse-double-pass, fast pass-by-pass, and `optical_bench` controls have been consolidated. Loading an older JSON first migrates it to `simulation.multipass`; the form and solver then use that structure. Download the migrated configuration to save the updated format without carrying legacy fields into new operating points.

File controls, Setup JSON, and material libraries are above the workspaces. Parameter sweeps are an optional tool below the run button. Each workspace retains fast and detailed models, with advanced solver overrides in secondary expandable sections. Switching workspaces does not remove fields that are temporarily hidden.

After a JSON upload, the program checks the strict core schema, merges defaults recursively, and fills the form. Unknown core keys cause errors. Put plugin and future-version extension fields in `extras` or top-level `plugins`.

---

5. Importing JSON and Material Libraries

5.1 Setup JSON

A Setup JSON contains the optical paths, material, Pump, Seed, grid, and solvers for one operating point. A successful upload **automatically refills and refreshes the form**. Material-library records use a separate workflow: select an operating point, then click Apply.

The program:

- Fills newer fields absent from an older JSON.
- Preserves unknown `extras` and plugin fields.
- Checks units, ranges, and incompatible combinations.
- Shows the actual resolved settings in the effective solver-stack display.

For incompatible settings, the form shows the requested value, the effective value, and the reason for adjustment. Removing the constraint restores the original request for fields still managed by that constraint. Imported file units, constant phase, dispersion reference wavelength, nonuniform caustic-probe positions, and six-face ASE reflectivities survive workspace changes. Download the complete current configuration and check the effective model; screenshots alone do not retain these settings.

5.2 Material-Library XML/JSON

A material library and a Setup have different roles:

- **Setup:** apparatus and solver settings.
- **Material record:** intrinsic or calibrated material data.

When applying a material, choose:

- Material name.
- Actual doping percentage.
- Operating temperature.
- x/y/z polarization projection.
- Whether to apply spectra, Sellmeier data, database heat fractions, and related entries.

If the database is normalized to 1% doping,

$$N = N_{1\%} \times C_{doping}.$$

Pump absorption-coefficient curves are first converted to cross sections:

$$\sigma_{pa}(\lambda) = \frac{\alpha_{1\%}(\lambda)}{N_{1\%}}.$$

Those cross sections then enter propagation with the actual ion density. The same α is not added a second time as passive absorption.

Applying the material updates:

- Ion density.
- Pump/Seed cross sections and spectral curves.
- Upper-state lifetime.
- Seed/Pump refractive indices.
- `dn/dT`.
- Sellmeier coefficients.
- Thermal conductivity, mass density, and specific heat.
- Available temperature/doping-response inputs.
- The material record's SHA-256 and a snapshot of the applied parameters.

The importer converts common Commercial App XML/normalized-JSON units to SI, including `ions/mm³→ions/m³`, `1/mm→1/m`, `mm²→m²`, `W/mm/K→W/m/K`, `N/mm²→Pa`, `g/mm³→kg/m³`, and `J/g/K→J/kg/K`. By default, material

snapshots reject out-of-range doping, temperature, Pump wavelength, or Seed wavelength, recording the decision in `material.library.snapshot_range_contract`. Only an explicit `snapshot_out_of_range="edge"` permits a reduced-order edge-value extension outside the range.

The solver does not automatically use data whose physical interpretation is ambiguous. Proprietary rate-line encodings for multiple dopants, complete nonequilibrium Stark networks, and 6×6 dimensionless strain-optic tensors remain in provenance/available-data records; they are not converted by assumption into scalar cross sections or stress-optic coefficients in Pa^{-1}. If material-library parsing fails, manually entered values remain unchanged.

5.3 Temperature-Surface Data

Material-library records may provide three levels of information:

1. Scalars at a fixed operating point.
2. A separable model combining $\sigma(T)$ with a normalized wavelength profile.
3. A full two-dimensional $\sigma(T,\lambda)$ table.

The importer uses only the information present in the record. Data are stored in `material.spectroscopic_response`; the default `mode=off` leaves the response disabled, so importing a material alone does not change previous results.

5.4 Creating a Traceable Material Snapshot

Material-library files are user-supplied; they are not included in this distribution. After providing a compatible library, the following example embeds its parameters in a new configuration. Replace `supplied-material-library.json` and the material/operating-point selections with your own data:

```
amplifier-sim materialize \
  configs/example.json supplied-material-library.json \
  --material 'Cr:ZnS' \
  --doping-percent 0.7 \
  --temperature-K 293.15 \
  --polarization x \
  --snapshot-out-of-range edge \
  -o configs/crzns_materialized.json
```

The output stores material parameters, the source-file hash, and the selected conditions for reproducible calculations and long-term retention. The default range policy is `error`. In this example, `edge` explicitly permits a reduced-order extension using legacy data's edge values. That choice is recorded in warnings/provenance and does not constitute interpolation within the working range.

---

6. Operating Modes, Steady State, and Solvers

6.1 Accuracy Presets

Preset	Main use	Default characteristics
`commercial_app_fast`	Broad Gaussian/Commercial App-style screening	Seed/Pump ABCD, steady-state populations, depletion and saturation, material-dependent thermal calculation, quadratic Kerr lens
`fast`	Rapid optical-path and trend checks	ABCD, algebraic steady-state populations, no thermal calculation, no saturation
`balanced`	General amplifier modeling	Seed FFT, conservative Pump ABCD, transient populations, depletion, saturation, material-configured thermal calculation

Preset	Main use	Default characteristics
`detailed`	Detailed three-dimensional modeling	Pump/Seed FFT, transient populations, 3D thermal calculation, and Fresnel interfaces

Presets provide default combinations. Explicit selections in the advanced area override them. `auto` continues to follow the preset.

6.2 Seed Solver Models

`auto`

When `physics.preset="commercial_app_fast"`, this resolves to `commercial_app_fast`. Otherwise, explicitly configured incoherent HG channels, multiple wavelengths, time resolution/time-file inputs, or Fourier-linked inputs select `spectral_time`; the remaining cases select `fluence`. `auto` does not independently upgrade to JOSA or coherent CPA. Select those models explicitly when needed.

`fluence`

The fastest monochromatic fluence model. Each crystal z cell uses the local Frantz-Nodvik saturation formula. It is suitable for quick estimates of energy, beam size, gain, and multipass behavior. It does not retain spectral phase or calculate pulse compression.

`spectral_time`

Array dimensions are time bins × wavelength bins × y × x. Time bins pass in front→peak→tail order and share populations; wavelength bins share the same stored energy. This captures gain narrowing, reabsorption, and pulse-envelope changes. Local saturation still uses FN. The default joint weight is the product of temporal and spectral marginals. `seed.joint_time_frequency.model="matrix"` accepts a nonseparable two-dimensional energy-weight matrix directly, such as a blue leading edge and red trailing edge. This remains a joint intensity distribution and does not retain complex coherence between frequencies. Coherent compression requires `coherent_cpa`.

`josa_transport`

This replaces the FN step in each z cell with JOSA B photon-density/inversion transport. Available schemes are:

- `explicit_cfl`: the paper's first-order explicit scheme, with `Δt=Δz/vg`.
- `pc_cfl`: the paper's predictor-corrector scheme, still subject to the CFL relation.
- `retarded_pc`: retarded time, $T = t - z/v_g$, so time bins are independent of Nz.
- `full_3d_pc`: fixes the three-dimensional phase-insensitive rate-model stack. Group-delay bin mixing uses an energy-conserving local-phase closure approximation. Diagnostics report this explicitly; it must not be interpreted as exact incoherent-channel propagation.

The first three schemes can fall back according to the selected policy. The solver first restores populations to their state before the Seed traversal, then recomputes with `spectral_time`/FN from that same state. A fallback selected by an initialization probe remains in use for the whole run. An isolated pulse may also return a result after runtime fallback.

If a backend changes during a stateful mapping-a finite pulse train, `pulse_map`, or a thermal-optical fixed point-that mapping cannot count towards convergence, and Anderson history is cleared. The solver fails explicitly if one consistent backend cannot close the final state. `full_3d_pc` raises an error rather than silently reducing fidelity.

`coherent_cpa`

Uses complex spectral fields and retains spectral amplitude, phase, and spatial wavefront. Supported features include:

- GD/GDD/TOD/FOD.
- Stretchers and compressors.
- Material dispersion.
- Kerr/SPM and B-integral.

- Optional front/peak/tail transient-gain soft windows.
- Multiphoton and free-carrier loss.

`seed.coherent.fidelity_mode` selects among configured, whole-pulse hybrid, axisymmetric Kerr, axisymmetric transient gain, and full reference transient-gain configurations. Axisymmetric paths resolve radial structure and a finite set of azimuthal modes. They share the CPA name but do not represent an arbitrary 3D complex field.

`axisymmetric.azimuthal_fidelity` may be `m0`, `m0_m1`, `m0_m1_m2_quadratic`, or `m0_m1_m2`. The m=0 model requires a centered, untilted circularly symmetric input. The m=1 terms retain decentering/tilt; elliptical and astigmatic inputs require m=2. For a single coherent field, the axisymmetric configuration requires input M^2=1 along both axes, `spatio_spectral.model="shared_source"`, and a symmetric adaptive local operator. Projection loss, azimuthal residuals, radial-edge energy, and population conservation still have to pass diagnostic checks. Refine radial, angular, and gain-frequency sampling separately, and compare against a full-field reference point.

`incoherent_hg` can use three radial modes, HG00/HG10/HG01: `axisymmetric_kerr`, `m0_m1`, `maximum_order_x/y`=1, `maximum_total_order`=1, and capacity 3. It supports equal ensemble M^2 on both axes from 1 to 2. Each mode propagates independently with unit M^2 and shares populations and total-intensity Kerr; Cycle Average uses the same modes. This path requires strong-field loss and time-window-by-time-window gain to be disabled. It supports external ABCD plus internal radial propagation. Other explicit HG combinations continue to use the full x-y backend.

`commercial_app_fast`

The Commercial App/Gaussian-duct backend reduces the transverse field to Gaussian q/second-moment states, retaining power, centroid, both beam radii, tilt, and quadratic wavefront. At every crystal z layer and wavelength it still reads the 3D populations, cross sections, and local gain, and updates populations for active absorption and stimulated emission. This makes it suitable for broad screening, but it does not retain the arbitrary spatial structure of a complex-field BPM model.

- `temporal_model="fixed_duration"`: closes peak intensity using the input effective pulse duration; it does not produce physically interpretable SPM broadening.
- `temporal_model="effective_area_spm"`: calculates SPM, spectral feedback, and B-integral with a one-dimensional effective-area envelope. If `strong_field_effective_area` is also enabled, total multiphoton loss uses the exact Kth-order intensity moment of a 2D Gaussian. Free carriers/FCA/carrier refraction use an energy-equivalent scalar state, without constructing a full `x-y-t` field.
- With `spectral_feedback` enabled, `spectral_feedback_bin_count` controls the number of coarse frequency bins feeding SPM-broadened spectra back into gain. `spectral_feedback_grid_mode="full_coherent_span"` covers the complete coherent FFT frequency range and captures newly generated Kerr sidebands. `seed_energy_support` covers only the input Seed's effective energy support, concentrating bins but requiring checks for leakage outside that support.
- `seed.spatial_modes.model="incoherent_hg"`: each HG mode advances its own Gaussian q and energy. Modes at the same wavelength first sum their local fluence; shared populations are updated once, and the output is divided according to input proportions. This closure requires `fixed_duration`. It is mutually exclusive with the effective-area strong-field closure, which applies only to a single Gaussian moment state. The form retains explicit HG modes and disables the latter closure.
- `include_relaxation_during_pulse=true`: applies only to non-CW operation, `fixed_duration`, and an actual energy-level network, using a finite pulse train or `pulse_map`. `gain_temporal_bin_count` (3-4096) controls the intrapulse population time bins.
- `thermal_lens.model` may be `sampled_phase` or `local_parabolic`. The latter requires positive manually specified fit radii. `quadratic_model="full_xy"` retains xy coupling; `axis_aligned` explicitly drops it. Strict mode limits the relative discarded coupling through `maximum_commercial_app_axis_coupling_ratio`.
- When `strong_field_effective_area` and `free_carrier.model="rate_equation"` are both enabled, `carrier_periodic_relative_tolerance` (default 1e−6) and `carrier_periodic_maximum_iterations` (default 32) control the across-pulse fixed point for each local scalar carrier state. Each it-

eration includes pulse generation/FCA updates and dark-time decay. This is distinct from the global 3D carrier-convergence condition in `coherent_cpa`.
- `beam_quality_model="projection_residual"` can accumulate higher-order residuals before each projection into an effective output-M² proxy.

Inspect the actual backend, temporal/gain bin counts, minimum Gaussian projection overlap, maximum B-integral, spectral-support leakage, projection fallback count, and effective output M² in `summary.signal.gain_transport`. If projection overlap falls appreciably below 0.99, hard apertures/higher-order thermal OPD matter, strong spatial gain guiding is present, or the target quantity is compressed pulse width/peak power, use `coherent_cpa` or a full-field model and perform grid-convergence checks.

6.3 Gain Source

Specified Small-Signal Gain

Use `energy_levels.gain_override_m_inv` and `energy_levels.saturation_fluence_J_m2`. This is useful for:

- Initial optical-path alignment.
- Reproducing papers that provide only $g_0 L$.
- Comparison with FN benchmarks.

This specified gain is not automatically inferred from Pump power.

Pump and Rate-Equation Gain

Set `gain_override_m_inv=null`. The program propagates Pump, solves the energy-level populations, and calculates gain from cross sections at every voxel and wavelength.

6.4 Multipass Organization

The unified multipass structure is `simulation.multipass`. Templates first compile to crystal encounters and inter-pass paths; the execution layer then selects:

- `execution_mode="causal"`: advances all active passes on an absolute time axis and handles competition through shared populations when the same pulse occupies the crystal in multiple passes simultaneously.
- `execution_mode="sequential"`: completes the same encounter/routing plan one pass at a time, for backends such as `fluence` and `commercial_app_fast` that do not retain this type of resolved temporal packet.
- `execution_mode="auto"`: selects causal execution when the resolved `spectral_time` configuration satisfies its requirements; otherwise selects sequential execution. This is recommended for new configurations.

The unified structure incorporates the geometry, losses, directions, delays, and backend overrides previously held in `counterpropagation`, `pass_mode/passes`, and `optical_bench`. The old solver modules and interface entry points have been removed. Older JSON is migrated before strict schema validation.

6.5 Multiphysics Coupling

`one_way`

The sequence is:

```
Pump → populations → thermal field/Δn → final Seed
```

Use this fast sequence for initial calculations. Seed does not alter the current Pump thermal solution unless Seed one-way postprocessing is selected.

thermal_optical_fixed_point

The iteration is:

```
Pump complex field → absorption/populations/ASE → heat → stress/Δn
        ↑                                                    ↓
        └──────────────── next Pump propagation ─────────────┘
```

The loop can also include local temperature-dependent cross sections and Seed heating. It requires FFT Pump propagation inside the crystal, a 3D steady-state thermal solution, and the `thermal_3d` refractive-index model.

Parameters and monitored quantities include:

- Minimum/maximum iterations.
- Relative tolerance.
- Relaxation factor.
- Residuals for active/passive Pump absorption, populations, temperature, `delta_n`, and end-face sag.
- With fixed-point Seed heating enabled, residuals for irreversible Seed heat power and output energy are also checked.

Safeguarded Anderson fixed-point acceleration is enabled by default. It accepts only bounded, finite-state candidates whose predicted residual decreases. If the next measured residual worsens substantially, history is cleared automatically and relaxed iteration resumes. If periodic steady state is enabled internally, the next thermal-optical iteration reuses the previous periodic-boundary populations as its initial guess rather than restarting from the ground state. Disabling acceleration retains ordinary relaxed iteration. Excessive relaxation can oscillate; too little relaxation slows convergence.

The outer loop reports convergence only when its residuals pass and the inner steady-state population solve or periodic map converges in both the current iteration and final closure. Populations from an inner solve that reaches its iteration limit may initialize the next iteration, but are not accepted as a physical solution. The outer residual is then treated as infinite and Anderson history is cleared. Inspect `population_solver_converged` and `nonconverged_population_iterations` to check this condition.

6.6 Heating from Irreversible Seed Loss

- `off`: reports losses only.
- `one_way_postprocess`: uses a Seed probe to calculate loss; updates temperature, stress, bulk refractive index, tensor photoelasticity, and end-face sag; then recomputes Seed once from the same pre-Seed populations and incoming carrier state to avoid double depletion. This is one Picard update, not a self-consistent solution.
- `fixed_point`: includes the average heat power from Seed losses in the thermal-optical iteration.

Kerr is a pure phase effect and does not enter the heat source. Multiphoton loss, free-carrier loss, and optional passive Seed absorption are irreversible heat sources. `material.strong_field.deposit_fraction_as_heat` is the effective fraction of multiphoton and FCA loss entering the cycle-averaged heat source. It does not imply explicit solution of carrier internal energy, recombination radiation, or transport. Result fields are `multiphysics.seed_irreversible_heat_W` and `seed_irreversible_heat_coupling`. Older `seed_nonlinear_heat_*` names remain archive-compatibility aliases because those fields historically also included optional passive-absorption heat.

6.7 Pump + Seed Periodic Steady State

Without Periodic Steady State

`population_dynamics=steady_state` solves the Pump-only CW steady state, without repeated Seed extraction. The final Seed shot uses those post-pumping populations.

For a bounded operating-window test, combine `simulation.pulse_train_initialization="cycle_average"` with `pulse_train_cycles=2`. The program first calculates an average-population initial guess, then executes two complete Pump-cycle→Seed-pulse mappings. The `2` counts only the two physical pulses after initialization, not internal cycle-average probes. `pulse_train_initializer` and `pulse_history` record the two parts separately. `auto` may use the same initializer when applicable. If upconversion/cross relaxation makes the average initializer inapplicable, the program falls back. An explicitly selected cycle-average initializer that would discard long-lived free-carrier memory fails during configuration checking; `auto` skips that initializer instead.

By default, a finite pulse train places its first Seed pulse after a complete repetition period. If an experiment begins with known populations at t=0 and its first pulse arrives earlier, explicitly set `simulation.first_pulse_delay_s`. The delay must be positive and no greater than one Seed period. It requires a positive repetition rate and `pulse_train_initialization="configured_initial_population"`, and cannot be combined with periodic steady state. `null` retains the default first-pulse-after-one-period convention.

`cycle_average`

Converts the Seed pulse train to average photon flux:

$$P_s = E_s f_{rep}.$$

Each self-consistent iteration propagates Pump and average Seed again, then solves steady-state populations. This is suitable for high repetition rates when each pulse causes only a small local population jump. Iteration counts do not represent physical elapsed cycles. The stopping criterion uses the true algebraic closure residual before multiplication by the relaxation factor; reducing relaxation changes stability and speed without silently loosening the physical tolerance.

The selected FN/JOSA/CPA solver then propagates one final pulse through the converged populations. Its energy is multiplied by repetition rate for average power. The interface compares this final pulse with the average probe.

`reduced_pulse_map`

`reduced_pulse_map` is an initialization option for `coherent_cpa`, not a separate reported steady-state solution. Select `method="pulse_map"` with `pulse_map_initialization="reduced_pulse_map"`. The legacy `method="reduced_pulse_map"` is accepted as an alias for that combination. The selected low-dimensional whole-pulse proxy first supplies an approximate cycle-boundary population; the configured full pulse map then continues until its own population, output, and any enabled carrier convergence checks pass. Proxy convergence alone is not sufficient, and the calculation does not stop after just one final CPA pulse.

- `model="phase_insensitive"`: a phase-insensitive whole-pulse proxy with controlled `temporal_bins`/`spectral_bins`.
- `model="axisymmetric_coherent_kerr"`: an axisymmetric coherent Kerr proxy over frequency×time×radius, requiring `seed.coherent.fidelity_mode="axisymmetric_kerr"`.
- `relative_tolerance` and `consecutive_cycles`: control the initializer's proxy fixed point, not the final full-map convergence requirements. Spatial/spectral/radial sampling needs separate convergence checks.

The reduced initializer is attempted once per top-level run. Later thermal-optical iterations warm-start from the preceding full-map state instead of repeating the proxy. If initialization fails, the solver records the reason and continues with the full map from the available starting state. The initializer is not used for JOSA or ordinary FN. Any required cross-pulse carrier memory must converge in the configured full map; it is not established by the reduced initial guess.

`pulse_map`

Repeats the full mapping:

```
one complete Pump cycle → one Seed pulse → new populations
```

Iteration continues until both cycle-boundary populations and output energy converge. It is slower but retains interpulse recovery and strong single-pulse depletion.

The default `pulse_map_initialization=auto` uses `cycle_average` to form a numerical initial guess when the longest population-memory lifetime spans many repetition periods. The strict `pulse_map` then corrects it. The average steady state only supplies the initial guess; it does not change the final fixed-point equations. Initialization is skipped automatically for upconversion/cross relaxation or direct gain override.

A low-memory Anderson method accelerates complete-cycle mappings. It returns to ordinary mapping if a candidate produces negative populations, violates local particle-number conservation, predicts the residual poorly, or gives a worse residual at the next evaluation. With acceleration, cycle counts describe complete-cycle map evaluations rather than a continuous sequence of physically elapsed pulses.

When `coherent_cpa + strong_field_time_domain + rate_equation` are all enabled, periodic carrier state supplies a third stopping condition. By default it compares FCA optical depth, carrier phase, and FCA absorbed energy produced in adjacent cycles. You can explicitly select the maximum relative change of the full carrier-density field instead. The local scalar carrier fixed point in Commercial App is not this global 3D state, so that selector is not displayed for Commercial App.

6.8 Individual Advanced Overrides

- Pump depletion.
- Gain saturation.
- Interface model.
- Population dynamics.
- Pump/Seed spatial propagation.
- Whether optical planes are saved.
- Seed 3D-data retention policy.
- VTK output.
- FFT backend and thread count.

Check the effective solver stack before running, particularly after uploading older JSON.

---

7. Crystal, Material, and Energy Levels

7.1 Crystal Geometry

Cuboid: `cuboid`

Enter the x width, y height, and z length. These dimensions create an actual material mask used by:

- Pump absorption.
- Seed gain and aperture.
- Populations.
- Heat sources.
- Cooling surfaces.
- 3D visualization.

Cylinder: `cylinder`

Enter diameter and z length. The cylinder is aligned with z, and its cross section is represented by a Cartesian-grid mask. Stair-step boundary errors require Nx/Ny convergence checks.

7.2 Material Optical Parameters

- `refractive_index`: Seed refractive index.
- `pump_refractive_index`: separate Pump refractive index; if null, uses the Seed/material index.
- `pump_passive_absorption_m_inv`: Pump loss that does not create useful excitation.
- `signal_passive_loss_m_inv`: passive Seed/ASE loss.
- `dn_dT_per_K`: thermo-optic coefficient.
- `n2_m2_W`: Kerr nonlinear refractive index.
- `reference_temperature_K`: reference temperature for thermal stress and `dn/dT`.

7.3 Three-Dimensional Doping Distribution

The base `ion_density_m3` may represent a volume average or a peak value. The standard interface provides two explicit profiles:

- `axis_symmetric_linear`: varies from the two surfaces toward the coordinate center along x/y/z for a cuboid. For a cylinder, `radial_xy` varies from the outer circumference to the central axis.
- `axis_monotonic_linear`: varies in one direction from the negative-coordinate end to the positive-coordinate end along x/y/z.

Both use one **Variation (%)** value:

- In the two-sided/centripetal mode, `+50%` makes the center 50% higher than the surface; `−50%` makes it 50% lower.
- In the one-way mode, `+50%` makes the positive-coordinate end 50% higher than the negative-coordinate end; a negative value increases in the opposite direction.

Multiple directions combine multiplicatively, followed by volume-average or peak normalization. The interface displays central xy/xz/yz slices and minimum, average, and maximum absolute concentrations in real time. Legacy JSON surface-exponential profiles, center profiles with a variation length, superposition strength, and floors remain computable but are not exposed in the standard form.

`normalization=volume_average` preserves volume-average ion density and total ion count as the profile changes. With `peak`, the base density specifies the peak value.

The same density field is used for initial populations, rate equations, Pump absorption, Seed gain, local lifetime response, and output. Zero-doped crystal cells remain zero. Mean upper-state fractions/inversion in progress displays and sweeps are volume-weighted only over positively doped crystal cells, excluding undoped end caps from the denominator.

Undoped or Weakly Doped End Caps

The front and rear ends may each have an end-cap length and relative ion concentration. `0%` means a truly undoped end cap; `20%`, for example, retains 20% of the core-region concentration. End caps are applied after all other profiles, so the original profile floor cannot raise their concentration again. They remain part of the same host crystal, with unchanged refractive index, thermal conductivity, and geometric boundaries, while active ion density, active Pump absorption, populations, and Seed gain change accordingly. The sum of front and rear end-cap lengths must be less than the crystal length.

7.4 Energy-Level Models

Two Levels

The ground/lower laser level and upper laser level form one transition pair. This suits simplified quasi-three-level or direct absorption-emission systems.

Three Levels

Pump excites the ground state to a Pump level, which rapidly relaxes to the upper laser level. The signal transition returns to the ground/lower level.

Four Levels

The model retains all four populations: ground, lower, upper, and pump. Pump excites the Pump level, which relaxes to upper. The signal transition transfers population to lower, followed by relaxation to ground. All four levels participate in the calculation.

Custom N Levels

You can define:

- Level names.
- Main Pump and Signal transitions.
- Arbitrary linear decay channels and branching ratios.
- Initial population fractions.
- Nonlinear reactions such as upconversion and cross relaxation.

A nonlinear-process fragment is:

```
{
  "reactants": {"upper": 2},
  "products": {"ground": 1, "transfer": 1},
  "rate_coefficient": 1e-24
}
```

The fragment defines one reaction. Declare names such as `transfer` in the custom level list before using it; the fragment alone is not a complete Setup. Coefficient units depend on total reaction order. Nonlinear population processes require transient/pulse-map evolution because the linear algebraic steady-state solver does not solve their nonlinear closure.

Nonlinear reactions may use `single_step` or `adaptive_substeps`. The latter subdivides a population time step according to the current maximum fractional reactant-depletion rate and recomputes upconversion/cross-relaxation rates at every substep. It is more robust for strongly nonlinear cases. A smaller `target_fractional_depletion` is more accurate but slower; `max_substeps` limits runtime. This option adds no work if there are no nonlinear reactions.

7.5 Cross Sections and Lifetimes

- `σpa`: Pump absorption cross section.
- `σpe`: Pump stimulated-emission cross section.
- `σse`: Signal stimulated-emission cross section.
- `σsa`: Signal absorption/reabsorption cross section.
- `τupper`, `τpump`, `τlower`: corresponding lifetimes.
- `pump_quantum_efficiency`: the fraction of actively absorbed Pump photons that creates the target excitation. This is the transfer efficiency from Pump to the target level; do not enter a fluorescence-quenching fraction already represented by the total lifetime again.
- `upper_state_radiative_lifetime_s`: independent radiative lifetime. The total lifetime may include nonradiative quenching. The explicit radiative lifetime may be shared by ASE and the lifetime-based nonradiative-heating calculation.

Three-column CSV format:

```
wavelength_nm, sigma_abs_m2, sigma_em_m2
```

Pump and Signal curves can be uploaded separately. The solver interpolates at each discrete wavelength.

---

8. Pump Module

8.1 Meaning of Pump Power

`power_W` can represent CW/average power or waveform peak power, according to `temporal.power_mode`. For CW operation, power is constant throughout the cycle.

Pump's repetition time axis shares the system repetition rate. That rate is edited in the Seed area but stored in `simulation.repetition_rate_Hz`.

Active Pump absorption has two explicit transport definitions:

- `absorption_transport.model="population_dependent"` (default): calculates local net active absorption from each voxel's populations and Pump absorption/emission cross sections. Pump saturation therefore changes propagation depletion.
- `fixed_active_coefficient`: advances active Pump attenuation with a nonnegative scalar `fixed_active_coefficient_m_inv`. This is useful for Commercial App-style cases that explicitly disable Pump saturation. The fixed coefficient replaces only the propagation-depletion coefficient and its absorbed-power accounting. Ion excitation still uses the actual Pump photon flux, transition cross sections, and quantum efficiency; the coefficient must not be interpreted as a cross section.

Both modes separately add passive absorption and record the actual `absorption_transport_model` and fixed coefficient, if used, in Pump statistics.

8.2 Transverse Spatial Distribution

First choose a Pump source-parameter definition:

- `waist_m2`: enter x/y $1/e^2$ waist radii and M^2 directly.
- `waist_divergence`: enter x/y $1/e^2$ waist radii and maximum divergence half-angles; M^2 follows from the second-moment relation.
- `emitter_na`: enter the incoherent emitter-face radius and x/y NA. The program uses `alpha=asin(NA/n)` and `M²=pi*n*r*tan(alpha)/lambda0` to create a second-moment proxy for field propagation, where `lambda0` is the vacuum wavelength.

`super_gaussian_exponent` is the complete intensity exponent p in `exp[-2(r/w)^p]`; `p=2` is Gaussian. JSON, form, preview, and solver share one normalized definition rather than converting it repeatedly in different modules.

`geometric_efficiency` ranges from 0 to 1 and accounts for measured or unmodeled upstream losses before light enters the modeled path. It scales emitted power; the result ledger retains nominal power, effective source power, and efficiency. Stochastic ray tracing separately calculates element, interface, aperture, crystal-interception, and side-leakage losses. Fitting those same losses with `geometric_efficiency` would count them twice.

All three source definitions are normalized at the same configuration layer; JSON, interface, live preview, and final solver do not each repeat the conversion. Source definition and propagation algorithm have different roles. `emitter_na` retains native emitter radius and NA, which directly define stochastic-ray phase space in `stochastic_raytrace`. `super_gaussian_abcd` propagates beam size with the second-moment M^2 below while preserving the SG intensity shape. Ordinary ABCD/FFT field solvers use an M^2 field proxy.

For `waist_divergence` and `emitter_na`, the estimate is

$$\alpha = \arcsin(NA/n), \qquad M^2_{raw} = \frac{\pi n r \tan\alpha}{\lambda_0}.$$

`waist_divergence` substitutes the entered maximum divergence half-angle for α. Paraxial propagation does not permit $M^2 < 1$, so transport uses $\max(M^2_{raw}, 1)$. The interface and diagnostics retain the unclamped raw value to reveal inconsistent radius, NA, medium index, or units. In `emitter_na`, the radius always means the emitter-face radius, not a known coherent waist. Diagnostics list native ray parameters and field-solver fallback separately.

The transverse-center definition also depends on the model. For `emitter_na + stochastic_raytrace`, `center_x/y_m` is the emitter-face center at `z=0`. For `waist_m2`, `waist_divergence`, and field-propagation proxies, it is the optical-axis center at the caustic/equivalent waist. The interface changes labels for the current combination; the same numeric center cannot be transferred between these definitions without checking its meaning.

Supported profiles and attributes include:

- Gaussian.
- Super-Gaussian.
- Approximate top-hat.
- Annular/ring.
- Elliptical x/y radii.
- x/y M².
- x/y orders.
- Decentering, rotation, and incidence angles along both axes.

A waist normally means the profile-scale intensity $1/e^2$ radius; `emitter_na` instead uses the emitter-face radius defined above. For an ideal Gaussian, the D4σ radius equals the $1/e^2$ radius, and its D4σ diameter is twice that value. Super-Gaussian, top-hat, and ring profiles require their own second-moment conversion; the Gaussian equality does not apply to all of them.

`profile="super_gaussian"` uses an x/y-separable profile; `radial_super_gaussian` uses a radially elliptical profile. They agree in the Gaussian limit but differ in shape and D4σ conversion at higher order. The Seed form also offers the radial option.

8.3 Reference Plane for `focus_z_m`

- `prescribed_caustic`: measured from the crystal entrance and specifies only the internal caustic. Lenses before the crystal do not enter the final Pump calculation.
- `abcd` / `super_gaussian_abcd` / `hybrid_abcd_fft` / `fft`: measured from the Pump LASER source plane. Pre-crystal distances and lenses participate in propagation.
- `stochastic_raytrace + emitter_na`: rays are defined directly by positions and NA at the `z=0` emitter face; `focus_z_m` does not enter ray sampling. A waist-type source proxy instead follows its corresponding caustic/waist definition.

Check the reference plane when optical paths disagree. The interface labels this field according to the selected propagation mode.

8.4 Pump Propagation Models

`prescribed_caustic`

Calculates internal radii directly from waist, M², and focus. It is the fastest option, but does not propagate external elements or retain a full phase screen.

`abcd`

A Gaussian-envelope propagation backend. External Gaussian paths propagate analytically, retaining full power and q parameters. Inside the crystal it still maintains a Gaussian envelope, unlike sampled complex-field propagation in `hybrid_abcd_fft`.

Internal depletion uses a conservative closure: scalar remaining power plus a Gaussian envelope renormalized at each z plane. Each layer first computes active/passive Beer-Lambert absorption, then projects the remaining power onto the normalized envelope for the current ABCD radius, centroid, and orientation. Output reports absorption, transverse escape, and power-accounting residual separately.

The self-similar Gaussian shape is suitable for fast Gaussian operating-point calculations, but cannot resolve transverse hole burning from locally saturated absorption. Choose `hybrid_abcd_fft` or `fft` for depletion-induced transverse deformation, aperture diffraction, or 3D thermal aberrations.

super_gaussian_abcd

Maintains a normalized super-Gaussian intensity function and fixed SG exponent at every plane, propagating 1/e² radii, centroid, tilt, and quadratic wavefront with astigmatic q/chief-ray ABCD. In air, provided the M² floor is not triggered, it reduces exactly to the geometric-divergence relation `w²=w0²+(z−f)²tan²(alpha)`. SG=2 is exactly ordinary Gaussian ABCD.

For SG>2 it is a self-similar intensity-envelope closure, not an exact diffraction solution. Finite apertures and 3D thermal aberrations require FFT. For equal complete exponents `p`, it uses the radially elliptical function `exp[−2((x/wx)^2+(y/wy)^2)^(p/2)]`; unequal x/y exponents use a separable function. Both are normalized by analytical integration. JSON `order_i` is the half-exponent, so `p_i=2*order_i`.

Apertures and phase screens in this backend apply scalar throughput only, without reshaping the envelope. Three-dimensional thermal `delta_n` does not feed back into the Pump envelope, so this backend cannot be used for a thermal-optical fixed point.

hybrid_abcd_fft

Uses ABCD outside the crystal, reconstructs a complex Gaussian with curvature, decentering, and tilt phase at the crystal entrance, then uses FFT inside. This is suitable when the source-plane beam is large but only a small window is needed near the crystal.

fft

Samples a finite window at the source plane and propagates it segment by segment using FFT. It handles apertures, phase screens, and non-Gaussian deformation, but the window must cover the entire external optical path.

stochastic_raytrace

With a fixed random seed, this samples equal-power rays from source-plane transverse and angular distributions. For `emitter_na`, `auto` resolves to `uniform_na`: uniform sampling in the direction-cosine ellipse, corresponding to constant radiance from a planar source. You can explicitly choose `uniform_solid_angle` or a generalized Gaussian restricted to the NA support.

Rays pass sequentially through the configured Pump elements. Interfaces apply Snell/Fresnel calculations ray by ray; lenses and curved surfaces use paraxial ray formulas. Inside the crystal, ray slope controls subdivision of each z cell. Deposition uses path length rather than encounter counts.

The generalized Gaussian angular distribution uses complete intensity exponents `angular_super_gaussian_exponent_x/y`; 2 means Gaussian, and x/y may differ. These exponents change sampling weights within direction-cosine NA support without changing the maximum angular support specified by `emitter_na`.

$$\bar{I}_{cell}\, V = \frac{P_{in}(1 - e^{-\alpha ds})}{\alpha}, \qquad P_{abs} = P_{in}(1 - e^{-\alpha ds}).$$

Thus rate-equation intensity, active/passive absorption, and ray-power depletion are consistent in the same discrete step. As $\alpha \to 0$, the calculation uses the $P_{in} ds$ limit. Output reports crystal-entrance interception, external-element losses, side leakage, low-power ray termination, output power, energy-closure error, ray-segment samples per cell, and a statistical-noise proxy.

`pump.raytrace.deposition_model` defaults to `nearest_cell_track_length` to preserve existing JSON behavior. `bilinear_track_length` uses the same conservative bilinear weights to interpolate absorption/thermo-optic gradients and deposit intensity/heat, reducing jumps as rays cross cells. Neither model performs exact ray-segment intersection with every voxel, so both require ray-count and grid-convergence checks.

Use this path for incoherent, high-M² end pumping when the emitter face and NA are known. The solver uses sequential paraxial rays; it omits nonsequential multiple reflections in side-pumped cavities, scattering/rough surfaces, ray-level polarization evolution, and arbitrary CAD surfaces. Diagnostics state these limits. It is not a complete Zemax-level nonsequential tracer.

Start alignment with 10,000-100,000 rays, then check convergence using at least two random seeds and four times as many rays. A warning appears when more than 10% of populated deposition cells have fewer than

30 effective weighted ray-segment samples. Effective sample count uses squared actual path contributions; bilinearly splitting one segment among four cells does not count as four independent samples. Increase ray count or reduce deposition-grid resolution rather than interpreting Monte Carlo granularity as real hot spots.

8.5 Pump Temporal Waveforms

Supported definitions are:

- `cw`.
- `gaussian`.
- `rectangular`.
- `super_gaussian`.
- File input.

Gaussian and super-Gaussian temporal widths mean **intensity FWHM**. For any `order>=1`, the program uses `exp[-ln(2)|2(t-t0)/FWHM|^(2*order)]`. Order changes the wings/flatness without silently changing the FWHM.

A typical file has:

```
time_s_or_relative, relative_power_or_W
```

With `pump.temporal.value_mode="relative"`, the curve is normalized to `pump.power_W` according to `power_mode="average|peak"`. With `value_mode="absolute_W"`, the second column directly specifies instantaneous watts and is not scaled by `pump.power_W`.

`file_period_contract="zero_outside"` sets power to zero outside the file's range. `cover_period` requires the file to cover the complete actual window `[0, period]`: `period=1/repetition_rate_Hz` in periodic operation, or `pump_duration_s` in single-shot preparation. Otherwise the file is rejected. An absolute-W file must actually be sampled by transient populations or periodic steady state; an unsampled CW path cannot treat it as a constant.

For analytical waveforms, `power_mode="peak"` means the continuous waveform's peak. Rectangular pulses store bin-average power according to each time bin's overlap with the pulse interval, without renormalizing to the maximum sampled value. For example, a 10 W peak rectangular pulse lasting 25 ms and lying entirely inside a 1 s window still has 0.25 J input energy with only eight bins. Gaussian/SG waveforms use bin-center samples, so narrow peaks still require refinement.

`average` normalizes the discrete-window average power to `power_W`. For relative-power files, `peak` normalizes the interpolated samples by their sampled peak. Reported waveform `peak_power_W` is the sampled peak and may be lower than the configured continuous peak for coarsely binned rectangular pulses.

At positive repetition rates, analytical pulse-center delays are reduced modulo the period. Single-shot preparation has no wraparound. File waveforms are interpolated on their own time axes; analytical-waveform `delay_s` does not necessarily shift a file.

8.6 Time Samples per Cycle

`pump_time_steps_per_cycle` specifies how many Pump time slices divide one repetition period. For CW operation, 80 rather than 40 does not mean twice as many Pump pulses; it subdivides the same constant-Pump period more finely. Increasing it usually offers no benefit for a purely CW steady state. It increases temporal resolution for pulsed waveforms, interpulse relaxation, or pulse-map evolution.

Single-shot preparation uses `physics.population_dynamics="transient"` and `simulation.repetition_rate_Hz=0`. Set preparation duration with `simulation.pump_duration_s` and bin count with `simulation.pump_time_steps`. This path actually samples the configured CW/analytical/file waveform, preserving peak/average power definitions instead of forcing CW. With zero Pump, dark relaxation still advances over a nonzero preparation duration. This path does not use `pump_time_steps_per_cycle` and is not a periodic steady state.

8.7 Pump Spectrum

Monochromatic, Gaussian, Lorentzian, super-Gaussian, and file spectra are supported. The form shows spectral order for super-Gaussian spectra. At every order, `bandwidth_fwhm_m` is the exact spectral-density FWHM, not a scale parameter valid only in the Gaussian case. Each spectral bin uses its own photon energy, cross sections, and propagation response. Pump spatial coherence still follows the configured scalar/complex-field model.

8.8 Separate Pump Optical Path

Pump and Seed paths are completely separate. A lens in the Pump path affects only Pump; elements in the Seed path do not affect Pump. The right-side preview draws both beams together, but emphasizes the edited beam's element information.

8.9 Cell-Averaged Pump Intensity and Absorption

Pump active and passive absorption are deducted cell by cell along z. When the total absorption coefficient α is constant within a finite-volume cell, the rate equations use the Beer-Lambert **cell-averaged intensity**:

$$\bar{I} = I_{in}\frac{1-\exp(-\alpha\Delta z)}{\alpha\Delta z},$$

rather than the entrance intensity I_{in}, which has not yet undergone absorption in that cell. The program obtains the average by dividing local absorbed power density by α, consistent with the energy ledger for that step. It returns to I_{in} as $\alpha \to 0$. This definition is used on uniform main grids, independent optical-grid mappings, and in thermal-optical iterations, reducing systematic overestimation of inversion and output on coarse Nz grids.

---

9. Seed module

9.1 Single-pulse energy and repetition rate

Specify:

- The single-pulse energy E_s.
- The repetition rate f_{rep}.
- The interface displays the input average power $E_s f_{rep}$ as you edit these values.

Output average power equals the final reported pulse energy multiplied by the same system repetition rate.

9.2 Time-domain, frequency-domain, and dual-domain input

Time-domain definition

Choose a Gaussian, sech², rectangular, or super-Gaussian pulse, or import a time-domain file. The time domain is the primary input. If no temporal phase is supplied, a flat phase is assumed. The program uses the convention in Section 9.6 to generate a read-only spectral preview.

Frequency-domain definition

Choose a monochromatic, Gaussian, Lorentzian, or super-Gaussian spectrum, or import a spectral file. The frequency domain is the primary input. If no spectral phase is supplied, a flat phase is assumed. The program uses the convention in Section 9.6 to generate a read-only temporal preview.

Dual-intensity constraints

When temporal and spectral intensities are both supplied without phase, the program uses alternating projections to find a compatible complex field. The solution is generally not unique, so it cannot replace FROG, SPIDER, or d-scan measurements.

An imported configuration may also retain `seed.pulse_definition.domain="independent"`: the temporal and spectral intensities are described independently, including a valid monochromatic spectrum. The two marginal distributions are not known to describe the same Fourier-consistent complex field. Loading this mode

into the form does not convert it into a Gaussian broadband input. To recover the actual phase, provide measured constraints or explicitly select a complex-field input definition.

9.3 Input file formats

Time-domain files:

```
t_fs, intensity
t_fs, intensity, phase_rad
```

Spectral files:

```
wavelength_nm, intensity
wavelength_nm, intensity, phase_rad
```

Intensities may be normalized. Absolute spectral-energy input can use J/bin or J/nm; the program integrates the spectrum and updates the single-pulse energy.

Specify table units with `time_unit` and `wavelength_unit`. Supported time units are `s/ms/us/ns/ps/fs`, and supported wavelength units are `m/um/nm`. The same conversions apply to Pump, Seed, coherent complex fields, separate phase files, and cross-section tables. Unknown units are rejected before propagation; the program does not assume they are SI. `energy_per_bin_J` denotes the integrated energy of each native sampling bin. To preserve the spectral shape, conversion to a uniform frequency grid first divides each bin's energy by its native frequency-bin width, including for nonuniform samples.

9.4 Temporal FWHM and spectral FWHM

The time-bandwidth product uniquely relates these widths only when the pulse is transform limited and its shape and phase are known. Because actual inputs may include GDD/TOD or a non-Gaussian spectrum, the program allows the widths to be specified independently.

Both the temporal and spectral widths of analytic super-Gaussian inputs use `exp[-ln(2)|2u/FWHM|^(2*order)]`. The entered width is therefore the exact intensity or spectral-density FWHM at every order. Changing the order does not introduce a width conversion factor. For file inputs, FWHM is determined by the sampled curve.

With `spectral_density_domain="wavelength"`, `bandwidth_fwhm_m` is the wavelength-density FWHM symmetric about the peak wavelength. With `spectral_density_domain="frequency"`, entering `bandwidth_fwhm_Hz` directly is preferred. Alternatively, specify only `bandwidth_fwhm_m`: the program retains `wavelength_m` as the peak wavelength and preserves the **total** FWHM after mapping back to wavelength coordinates, but the two half-maximum wavelengths will not be symmetric about the peak. `bandwidth_fwhm_m` and `bandwidth_fwhm_Hz` cannot both be positive.

Supplying only spectral intensity and assuming flat phase produces the transform-limited temporal waveform associated with that spectrum; it need not be Gaussian.

9.5 Fourier sampling and solver bins

- `seed.pulse_definition.fft_sample_count`: the uniform Fourier grid used to construct and preview the complex pulse.
- `seed.spectrum.sample_count` and `span_fwhm`: the initial discretization of the analytic Gaussian, Lorentzian, or super-Gaussian **source spectrum**. The displayed span is the number of FWHM widths covered on each side of the center.
- `seed.pulse_definition.solver_time_sample_count`: temporal transport bins for Fourier-linked input.
- `seed.pulse_definition.solver_spectral_sample_count`: wavelength-dependent gain bins for Fourier-linked input. If these two fields are absent, each uses the legacy `solver_sample_count` fallback.
- Independent input uses `seed.temporal.sample_count` and `seed.spectrum.sample_count` for temporal and spectral discretization.

- `seed.coherent.sample_count`: the requested number of complex spectral samples for independently constructed CPA input. Fourier-linked input can use the actual `seed.pulse_definition` grid directly, so consult the sample count in the run diagnostics.

Each sample count controls a different stage; increasing one does not replace the others. In particular, Commercial App spectral feedback requires the source spectrum itself to contain multiple effective samples. Increasing only `spectral_feedback_bin_count` cannot turn monochromatic or zero-bandwidth input into a broadband source. When increasing `span_fwhm`, also increase the source-spectrum sample count. Check convergence separately for source discretization, transport rebinning, and feedback bins.

Accuracy also depends on sufficient temporal and spectral coverage, not just the sample count. Too few samples miss peaks or narrow structure; too many substantially increase the FFT and gain-update cost of every z step.

The Fourier grid includes broad padding regions with nearly zero energy to support temporal sampling. Before forming physical wavelength-resolved solver bins, the program trims numerical tails with a combined cumulative energy of no more than approximately `1e−12`, then normalizes and rebins the remainder. This prevents low-frequency padding from extending the wavelength axis by several micrometers or consuming the limited wavelength bins. Coherent CPA itself retains the full FFT grid.

The Fourier spectral preview does not directly plot integrated energies from uniform frequency bins against a nonuniform wavelength axis. It maps the frequency-bin boundaries to wavelength widths and displays wavelength spectral density in `J/nm`. Its integral and FWHM therefore both refer to wavelength spectral density.

9.6 Spectral phase

The common temporal convention is `E(t)=Σ E(ω) exp(−iωt)`, with spectral propagation phase multiplied by `exp(+iφ)`. Under NumPy's sign convention, **spectrum-to-time uses FFT, and time-to-spectrum uses IFFT**, with `norm="ortho"` and the corresponding axis shifts. Spatial diffraction FFTs use their own coordinate conventions. Positive GD shifts the pulse to later times; with positive GDD, high-frequency blue components arrive later than low-frequency red components. Coherent diagnostics record `fourier_convention="exp_minus_i_omega_t"`. The same convention is used for input construction, phase retrieval, previews, CPU and experimental GPU calculations, and propagated temporal results.

Before importing a phase file, check the instrument's temporal sign convention, the frequency/wavelength ordering, and the reference wavelength. The program neither infers the convention nor conjugates the data automatically. Asymmetric pulse, chirp, and carrier-timing results obtained with an older sign implementation must be recalculated; changing axis labels alone is insufficient.

The phase is expanded about the reference angular frequency:

$$\phi(\omega) = \phi_0 + GD\,\Delta\omega + \frac{GDD}{2}\Delta\omega^2 + \frac{TOD}{6}\Delta\omega^3 + \frac{FOD}{24}\Delta\omega^4.$$

- GD changes the time origin without changing pulse shape.
- GDD primarily produces second-order chirp and broadening.
- TOD produces asymmetric structure.
- FOD affects higher-order wings.

The Seed section provides separate controls for:

- Additional input-field phase.
- ZnSe/stretcher phase.
- Compressor phase.
- A separate phase file or phase columns within intensity files.

`constant_phase_rad` and `reference_wavelength_m` are retained in the configuration. Changing the Seed center wavelength does not automatically overwrite an explicitly specified dispersion reference wavelength. If both intensity and phase were measured after the ZnSe, do not add the same ZnSe GDD again. If only intensity is available, specify how the missing phase will be supplied.

9.7 Spatial beam, M², and focus

The Seed supports separate x/y waist radii, M², super-Gaussian orders, offsets, rotation, wavefront curvature, incidence angles, and astigmatic focal positions `focus_x_m/focus_y_m`.

M² specifies the diffraction rate of the beam's second moments; it does not determine the actual transverse-mode phase. The optional `hermite_gaussian` maximum-entropy approximation constructs an incoherent HG intensity mixture from M² and the maximum mode order. It preserves second moments and is a fast scalar reduction. `incoherent_hg` uses the same maximum-entropy weights but propagates every HG-mode/wavelength combination as a separate synchronized channel. Each mode independently experiences apertures, diffraction, local gain, and thermal phase. All channels share the population in the same time/z cell and are summed incoherently at the end.

Maximum HG mode order

`maximum_order_x/y`=N includes $HG_0 \ldots HG_N$ on each axis. The finite mode set can represent

$$1 \leq M^2 \leq 2N + 1,$$

so at least $N \geq \lceil (M^2 - 1)/2 \rceil$ is required. The interface raises the minimum allowed order to this bound. Increasing the order further permits a broader maximum-entropy weight distribution; it does not change the requested M². The x/y weights are solved independently and combined as a Cartesian product, with incoherent addition between modes.

`incoherent_hg` also supports `maximum_total_order`, propagating only modes satisfying `n_x+n_y≤N_total`. `null` selects the full x/y Cartesian product. The total-order truncation must still represent the specified M² on both axes, and the final mode count must not exceed `maximum_propagated_modes`. These limits set the computation budget. The program does not discard modes silently to fit memory.

Under ② **Beams → Seed → Spatial beam, M², and transverse modes**, select the option for propagating HG modes individually with shared populations, then enable the radial three-mode option for HG00, HG10, and HG01, using only m=0, ±1. The form displays total order 1, capacity 3, the actual nonzero modes, and their energy fractions. For M²=1.15 on both axes, the weights are 85%, 7.5%, and 7.5%. Azimuthal order m and total HG order are different quantities; enabling three modes does not enable azimuthal m=2. Disable the option to return to full x-y propagation.

Changing M² in the form leaves the source waist radii and focal positions unchanged, so the minimum waist in air after the lens is not held fixed. To hold that waist fixed, back-calculate the source-plane q parameters from the optical system, then enter the corresponding source waist radii and focal positions.

`effective_m2` does not construct an explicit HG mode set; it uses M² only in the Gaussian/q-parameter diffraction law. None of the three approaches can uniquely reconstruct unmeasured mutual intensity or actual mode weights from M² alone. `incoherent_hg` provides propagation of individual modes, not a unique reconstruction of the input. When a sampled complex field is retained, output M² is recalculated from its spatial/angular-spectrum second moments instead of copying input M². Uniform grids use spectral derivatives, and nonuniform grids use finite differences. If inadequate sampling drives the calculated value below the physical lower bound, the result requests a larger or finer optical window.

The effective-M² scalar FFT uses `lambda_eff=M²*lambda0`. First-order geometrical phases-including source curvature, tilt carriers, thin/thick lenses, and curved mirrors-therefore use the same effective wavenumber, keeping the FFT focus and chief ray consistent with ABCD propagation. Thermal, Kerr, and user `phase_screen` terms remain physical phases defined at the actual wavelength. Applying these phases to a single scalar field with `M²>1` is an engineering surrogate rather than rigorous propagation of mutually incoherent modes. For quantitative studies of high-M² beams through strong aberrations, use an individual-mode/mutual-intensity model, or at least cross-check independently against `M²=1` and the experimental caustic.

9.8 Seed spatial gain closure

The **Seed gain transport** controls display the temporal backend and spatial closure together, but they are separate settings:

- `local_field` is the default fully sampled field model. Each x/y cell uses its own active gain and passive loss, allowing gain guiding and combination with spatial FN saturation, wavelength-resolved transport, JOSA, CPA, and non-Gaussian beams.
- `gaussian_mode_overlap` is a constrained Gaussian-amplification comparison. Within each z cell, a half-step of diffraction and thermal phase is followed by integration of the three-dimensional local gain against the current Gaussian intensity to obtain one effective gain. Only the total beam power changes. The subsequent second-moment projection preserves power, centroid, both radii, tilt, and quadratic wavefront while discarding higher-order aberrations and gain guiding.
- `parabolic_gain_fit` fits the actual three-dimensional active gain to an axis-aligned quadratic gain duct within an ellipse defined by `fit_radius_x/y_m` at each z layer, then applies it to the complex field. Outside the ellipse, the original local gain is used. `minimum_gain_m_inv` clamps only the fitted values inside the ellipse and may be negative when genuine reabsorption is present. This implements the Commercial App Gauss-BPM spatial closure without changing Pump, level populations, cross sections, lifetimes, or the input beam.

`gaussian_mode_overlap` currently requires monochromatic `fluence`, `gain_saturation=false`, a Gaussian or first-order super-Gaussian beam, and `effective_m2` or pure HG00. Axis-aligned ellipses and circular beams at any orientation are allowed. Rotated ellipses, higher HG modes, wavelength-resolved input, JOSA, and CPA require `local_field`. These restrictions exclude combinations for which array broadcasting works but the physical coupling is undefined.

`parabolic_gain_fit` currently requires a CW Seed and a resolved backend of `fluence` or `commercial_app_fast`. The `fluence` cycle-averaged steady state uses the fitted stationary-photon-flux closure; Commercial App retains its own Gaussian-q transverse state. The fitting ellipse must contain enough active grid cells. Its center, radii, minimum gain, and layer-by-layer residuals are recorded in the diagnostics.

`record_axial_diagnostics=true` saves input/output energy or average power, effective active/passive/net gain, coherent Gaussian-projection overlap, and second-moment radii for every pass and z cell. The cycle-averaged steady state and final pulse use the same spatial closure; an inconsistent combination fails configuration checks. `summary.signal.gain_transport` also records the backend, whether gain guiding is allowed, the maximum cell gain exponent, safety clipping of positive exponents, minimum projection overlap, any subsequent field reshaping in the output optical train, and axial samples. Active extraction and passive loss share the same cell-averaged-field ledger. If safety clipping is activated, the ledger is scaled consistently so that ions do not release unclipped energy into an already clipped field.

Local thermal lens for sampled fields

The local thermal lens and spatial gain closure are separate operators. `seed.thermal_lens.model="sampled_phase"` directly uses the sampled three-dimensional thermal refractive-index phase. `local_parabolic` fits a local quadratic thermal lens inside manually specified elliptical radii at every z layer, then applies the fitted phase in the sampled-field propagation of `spectral_time`, JOSA, and `coherent_cpa`. `full_xy` retains the xy cross term; `axis_aligned` retains only the x/y astigmatic terms. If the fitting center is not specified, it follows the current beam center. Commercial App Fast does not read this root-level field; it uses `seed.commercial_app_fast.thermal_lens`. The fast `fluence` backend also does not consume the sampled-field local thermal lens. These controls appear only when the selected backend uses them.

9.9 Jones polarization and thermo-photoelastic birefringence

`seed.polarization.model="jones"` extends the monochromatic `fluence` field to two complex components, `Ex/Ey`, in a fixed laboratory x/y basis. Diffraction, interfaces, and scalar gain act on both components separately. Local FN saturation and population depletion always use $|Ex|^2+|Ey|^2$, avoiding double amplification of the same energy. Linear, elliptical, and left/right circular input polarization are supported.

With `birefringence_model="radial_tangential_plane_stress"`, the field is rotated into the local radial/tangential eigenaxes in every crystal z cell, the following changes are applied,

$$\Delta n_{r,\theta} = 2n_0^3 \alpha_T C_{r,\theta} \left[T(r,z) - T(0,z) \right],$$

and the field is rotated back into the x/y basis. The photoelastic term uses the spatial temperature difference relative to the axis of each z slice, not the absolute rise above coolant temperature. Uniform heating therefore produces only the common `material.dn_dT_per_K` thermal phase, without spurious stress birefringence. The current implementation requires monochromatic `fluence`, `local_field`, `thermal_3d`, and `material.mechanical.stress_model="none"`. The last requirement prevents double counting of scalar `Δn_stress` and the Jones photoelastic term. This reduced expression applies only to approximately axisymmetric end pumping and isotropic mechanical boundaries; it is not a full mechanical-equilibrium FEM for rectangular crystals, clamping stresses, or arbitrary crystal orientations.

Results retain integrated Stokes parameters, parallel/orthogonal analyzer powers relative to the input polarization, complex-field second-moment M² for both analyzer channels, and `output_jones_field`. The orthogonal-analyzer fraction describes an ideal analyzer. The program does not automatically deduct it from total power. If experimental M² is measured after an output polarizer, compare with `parallel_analyzer_beam_quality` rather than the polarization-insensitive total-field M².

9.10 Spatial-spectral coupling

A broadband Seed can assign different values at different wavelengths for:

- Waist radii.
- M².
- Focal positions.
- Transverse centers.
- Incidence angles.

Available modes are:

- `shared_source`: all wavelengths share the source spatial parameters.
- `chromatic_parametric`: power-law/linear parameters based on wavelength offsets.
- `wavelength_table`: interpolation of a wavelength-by-wavelength table.

These settings describe the spatial part of $E(\omega, x, y)$. GDD/TOD instead describe spectral phase.

9.11 Coherent CPA and transient gain

Protected temporal-window tails (2026-09-14): enable the option for protecting the temporal soft-window tails, with one window on each side, in the Seed coherent settings. The central cumulative-energy coverage (%) defines equal-energy tails, not a peak-height threshold. The total window count includes the two tails: 99% coverage with 9 windows means 1 left-tail window, 7 central windows, and 1 right-tail window. Enabling the option fixes the total count with identical initial and maximum values; it does not automatically increase to 65. Depletion and reconstruction errors are still reported. The full FFT and complex-field tails are preserved, and tail windows extend to the temporal grid boundaries. For each pulse propagation, the interval is selected using the actual temporal energy at the first gain input, including broadening from preceding optics. It remains fixed for that propagation and is selected again for the next one.

The **optical and gain substeps per Nz cell** controls also support a front-to-tail gain-update limit. Whole-pulse mode uses independent sub-slices, while transient mode shares populations. An explicit update-count limit is not supported with intrapulse relaxation or carrier coupling that retains memory. With N optical steps and a gain limit G, the gain operator is called min(N,G) times, processing all temporal windows internally on each call. O1/G1 means one optical substep and one gain call containing every temporal window per Nz cell, not two half-step gain calls. The explicit limit takes precedence over automatic merging thresholds. Use the diagnostics to assess whether the chosen limit is accurate enough.

Coherent spectral-tail merging remains limited to whole-pulse axisymmetric gain. Front-to-tail mode uses the full frequency grid. The form can be configured for O1/G1, 99% cumulative energy, and 1+7+1 temporal windows. The separate research development tree includes corresponding synthetic fixtures and import/export re-

gressions in `tests/source_config_fixtures.py` and `tests/test_protected_transient_ui.py`; these development test files are not included in this distribution.

By default, CPA integrates over the entire pulse and applies local saturation to the total fluence. This is fast, but it does not resolve differences in depletion between the front, peak, and tail.

Enabling `coherent.transient_gain.enabled` performs these steps:

1. Transform to the time domain using the `exp(−iωt)` convention.
2. Divide the pulse into overlapping soft windows.
3. Return each window to the frequency domain and apply wavelength-dependent gain.
4. Immediately commit the population change caused by that window.
5. Use the updated populations for subsequent temporal windows.

`adaptive_slices` increases the window count when depletion within one window is too large. The default `reuse_adaptive_slice_floor=true` reuses an already validated lower bound on the window count across z layers during one pulse propagation. This avoids starting every local gain step with too few windows and retrying after failure. Depletion is still checked against the same acceptance criteria. Inspect the maximum depletion per window and the soft-window energy reconstruction error. Fixed window counts, including one window covering the full pulse support, must also satisfy the depletion gate. If the depletion gate is not met, the configured failure policy applies even when all fixed windows have been processed.

If CPA fallback is allowed, failure handling first restores the populations to their state before the entire Seed call, then reruns with fluence gain while retaining the coherent field, CPA dispersion, and Kerr effects. Diagnostics distinguish initialization probes from runtime failures using `fallback_origin=initialization|runtime`. Initialization fallback can define a consistent backend for an entire run, and an isolated pulse may also return a runtime-fallback result. A finite pulse train, periodic fixed point, cross-pulse free carriers, or thermal-optical loop cannot mix two backends partway through one state mapping. Runtime fallback cannot count as convergence. If one consistent backend cannot complete the final closure, the calculation fails instead of returning a state assembled from different backends.

The advanced local operator's `multirate_gain.enabled` option keeps fine substeps for dispersion, Kerr, thermal phase, and diffraction while reducing calls to transient soft-window gain. This approximation is disabled by default. When enabled, both a maximum gain exponent per update and a maximum number of merged substeps limit it. Intrapulse relaxation or strong gain automatically restores updates at every substep. Check the reduction in gain calls reported by `cpa_diagnostics.local_operator.multirate_gain`, then compare convergence with the option disabled.

Whole-pulse gain also supports `local_operator.spatial_population_update="independent_groups"`. Every spatial gain group starts from the same pre-pulse particle-number density in its parent slice and uses its own axial volume for local saturation and frequency competition. The optical field passes to the next spatial group without carrying forward the preceding group's population depletion. After all groups finish, their results are volume-weighted and committed once at the parent-slice boundary. The thermal field, Pump, and periodic iteration retain the parent grid. Old configurations default to `shared_cell` so that historical calculations retain their original shared-population behavior within each slice. This option is available only for symmetric propagation with `hybrid_whole_fluence` or `axisymmetric_kerr`. It cannot be combined with gain updates by temporal window.

`multirate_gain.maximum_updates_per_cell` is an optional **maximum number of gain calls per slice**, distinct from the earlier limit on optical steps merged into each update. With a limit of 2, optical step counts of 1/2/3 produce 1/2/2 gain calls. Three steps form groups of 2+1 with aggregation weights of 2/3 and 1/3. Whole-pulse update-limit mode requires independent spatial groups and the multirate switch; transient mode requires shared populations and the multirate switch. Pure multiphoton absorption with free carriers disabled-including 2PA and 7PA-is allowed, but absorption with a retained carrier history is not. This mode takes precedence over the automatic gain-exponent and group-size thresholds; diagnostics report whether the original gain-exponent threshold was exceeded. Omitting the field or setting it to null retains the original automatic grouping rules. Each group applies gain at its representative middle optical substep, choosing the earlier middle substep for an even-sized group. No extra optical substeps are added.

The optical `maximum_substeps=3` is a hard limit per parent slice. `maximum_nonlinear_phase_rad_per_substep=0.2` is the adaptive threshold for Kerr phase, while gain, dispersion, and thermal phase contribute their own subdivision criteria. If the physical changes require more steps, the program executes no more than 3 and retains the `substep_cap_active` diagnostic. Reaching this limit does not establish that the accuracy requirement has been met.

9.12 Independent Seed optical path and radius probes

Radius probes accept accumulated optical-path positions referenced to either:

- This beam's LASER source plane as zero.
- The shared crystal entrance as zero.

The probes return x/y D4σ radii, centroids, upper/lower envelopes, and actual coordinates. Pump and Seed use their own source-to-crystal distances and offsets; the two are not mixed.

The **optical-path preview** at the top of the interface is a cold-state q-parameter preview before calculation. It applies the configured static material refractive index and sequential optical elements, but not the unsolved temperature field, thermal-optical Δn, gain guiding, or FFT aperture reshaping. After a calculation, the result's **3D optical-path workbench** displays both: thin solid lines/translucent beam volumes show the cold preview, and thick dashed lines/open circles/elliptical rings show the actual propagated D4σ. The 3D view is displayed first and can be rotated by dragging. The 2D envelope, pass energies, and recorded optical planes remain in the same result workspace. Inside the crystal, transverse second moments are automatically calculated for every optical z bin; only scalar diagnostics are added, without copying every complex-field slice. Pump values are cell-averaged intensities plotted at bin centers; Seed values are fluences after completing that bin's split step, plotted at bin exits. Separate entrance/exit markers retain the exact boundary planes. Disabling optical-plane recording still retains internal z-bin diagnostics, but does not save additional optical-plane markers such as source planes, boundary planes, and system output.

When Pump and Seed overlap, the selected beam is drawn last so that the Seed is not hidden by the translucent Pump surface. Only the drawing order changes; propagation is unaffected.

---

10. Multipass systems and optical benches

10.1 Unified multipass configuration

All new multipass setups use `simulation.multipass`. Fast multipass, counterpropagating double-pass, and custom benches now use this same configuration. The main fields are:

- `enabled`: enables multipass; otherwise the calculation is single-pass.
- `template`: `double_pass`, `two_mirror`, `herriott`, `relay_4f`, or `prescribed`.
- `pass_count`: the number of crystal hits.
- `execution_mode`: `auto`, `causal`, or `sequential`. Prefer `auto` for new configurations.
- `per_transition_power_transmission`: a common power transmission applied in addition to each interpass route.
- `per_transition_extra_delay_s`: a common extra delay beyond each geometrical route.
- `maximum_time_bins`, `include_dark_relaxation`, and `maximum_dark_step_s`: budgets for the causal clock and dark evolution.

All templates compile into a common plan of `hits` and `transitions`. `causal` advances all active passes on a global clock. `sequential` completes the same plan pass by pass, allowing backends such as `fluence` and `commercial_app_fast` to reuse hit directions, interpass elements, losses, and delays without a separate multipass system. New interface setups use only the unified templates. Imported legacy fast-multipass configurations are shown in their migrated sequential form; the old entry points are not restored.

`optical_train.seed` describes only the common input path up to the first crystal hit and the common output path after the last hit. Propagation between hits belongs to the chosen template or `prescribed.transitions` and must not also be duplicated in a global interpass optical train.

10.2 Causal finite multipass propagation and automatic overlap

If the external return time is shorter than the pulse envelope, so that successive passes can occupy the crystal simultaneously, select `signal_model="spectral_time"`, resolve the temporal pulse packet, and use `execution_mode="causal"`, or let `auto` choose it. The program retains every active pass channel on one absolute time axis and uses the first-order upwind CFL time step inside the crystal:

$$\Delta t = \frac{n_g \Delta z}{c \cos\theta}.$$

All pass fluences at the same time in the same crystal cell participate in one local saturated-gain event and update the same three-dimensional populations. Passes that overlap in time but have displaced transverse spots compete only in cells where they actually intersect. There is no switch to force overlap or nonoverlap.

Five template types are provided:

- `double_pass`: specify distance `D` from the rear crystal face to the return mirror; the route `D → mirror → D` is generated automatically. Enter mirror reflectivity, x/y curvature, center, and clear aperture, plus separate crystal-hit centers and local angles for the first and return passes.
- `two_mirror`: one mirror on each side of the crystal, used alternately. A finite pass count must explicitly use `ideal_switch` or `geometric` extraction; an ordinary passive cavity is not silently treated as exactly N passes.
- `herriott`: mirror separation, crystal position, curvatures/offsets/apertures, and an injected chief ray generate a paraxial spot trajectory. The finite sequence assumes extraction through a geometrical exit hole after the last pass.
- `relay_4f`: the object plane, two focal lengths, lens separation, and return-mirror distance generate a unit-magnification `B=0` round-trip relay. Per-pass center/angle increments represent a mirror array or a prescribed sampling sequence.
- `prescribed`: directly specify `hits` and `transitions` when a paper supplies only pass spots, angles, optical path lengths, or delays. The program labels this a boundary-condition model rather than claiming to reconstruct an unreported mirror layout.

`prescribed.transitions[].route_model` supports:

- `delay_only`: adds only `geometric_length_m × group_refractive_index / c` to the clock, leaving the sampled field unchanged.
- `free_space`: propagates the sampled field through the given length.
- `round_trip_mirror`: generates a free-space → mirror → free-space return route.
- `optical_train`: propagates through any supported sequential elements in `route`, suitable for custom folded paths and relays.

`extra_delay_s` represents delays not already included in the geometrical route. If the actual route changes the beam, `delay_only` cannot substitute for propagation.

Causal execution requires at least `signal_model="spectral_time"` and `seed.temporal.resolved=true`. The program checks the hit count against both `simulation.multipass.pass_count` and the template, so no second `simulation.passes` value is required. The current model treats the finite multipass evolution of one injected pulse. It does not treat separately injected neighboring pulses simultaneously remaining in the system, a coherently leaking cavity, or Maxwell-Bloch dynamics. Every pass shares one z-CFL clock. A relative spread in pass `cosθ` values above `maximum_relative_path_cosine_span` causes rejection rather than a silent approximation.

Geometrical group path and extra delay are each counted once. If a sequential backend already advances populations within the pulse, interpass dark relaxation subtracts the pulse-clock interval already advanced, avoiding duplicate recovery. Whether the intrapulse clock advances depends on the backend actually executed.

External delays in causal mode are rounded to the nearest CFL time bin. For quantitative results, compare at least two values of `grid.nz`. In `summary.signal.gain_transport.multipass`, inspect each segment's delay-quantization error, `compiled_hits`, overlap time versus z, three-dimensional spatiotemporal overlap fractions for each pair of passes, output energy, and pulse-shape convergence. A pass-pair fraction is defined as `Σ t∫xy min(Fi,Fj)/min(ΣtEi,ΣtEj)` and therefore includes both temporal and spatial overlap.

The separate research tree contains generic, non-paper-fitted Nd:YAG double-pass and eight-pass examples, named `configs/physical_causal_double_pass_ndyag.json` and `configs/physical_causal_multipass_herriott_ndyag.json`. They are not included in this release; the bundled `configs/example.json` is the supplied starting configuration.

10.3 Custom routes and migration of old configurations

Custom multipass systems use `template="prescribed"`. Each `hits[]` entry describes one hit on the same crystal, with independent direction, center, incidence angle, interface transmission, and related settings. Each `transitions[]` entry describes the route, loss, and delay between adjacent hits. A three-pass sequential example, `examples/prescribed_multipass.json`, belongs to the separate research examples and is not included in this release.

When loading legacy JSON, the program performs one deterministic migration before strict structural validation:

Legacy field	New configuration
Explicit return mirror in `simulation.counterpropagation`	`multipass.template="double_pass"` with causal execution
`counterpropagation.return_path_model="optical_train"`	`multipass.template="prescribed"`, with the original route moved into a transition
`pass_mode="optical_bench"`	Multiple `gain_medium` entries split into prescribed hits/transitions
Fast multipass with `passes>1`	An internal sequential template preserving direction, x flips, transmission, delays, and interpass elements

These mappings are used only when reading old files. There are no legacy counterpropagating double-pass or `optical_bench` solver branches at runtime, and the interface does not create legacy fields. After import, inspect the effective model and optical-path preview, then download the new JSON as the baseline for later work. A file that enables both old and new multipass definitions is rejected; neither definition is given an assumed precedence.

10.4 Optical elements

Element	Main fields	Function
`free_space`	`length_m`	Propagation in the current refractive index
`thin_lens`	`focal_length_m`, with separate x/y values if needed	Quadratic thin-lens phase or ABCD transformation
`double_pass_lens_relay`	Object-to-first-lens distance, two focal lengths, lens separation, second-lens-to-plane-return-mirror distance	An exact `B=0`, unit-magnification return operator from the full double-pass ABCD matrix
`thick_lens`	Curvatures, thickness, refractive index	Two interfaces and internal propagation
`aperture`	x/y radii or diameters, order	Field clipping, including actual energy loss with FFT
`interface`	`n_after`, model, angle, polarization	Ideal, coated, or Fresnel interface
`mirror`	Curvature/focal length, reflectivity	Reflection, focusing, and spectral reflectivity
`tilt`	x/y angles	Linear spatial phase
`coordinate_break`	Angles, offsets	A change to subsequent local paraxial coordinates
`phase_screen`	Phase/coefficients	User phase screen or aberrations
`dispersive`	GD/GDD/TOD/FOD	Spectral phase
`loss`	Power transmission	Loss without geometrical action
`observation`	Name	Saves a diagnostic plane
`gain_medium`	Direction, angles, interfaces	Defines one crystal hit

Before propagation, formal JSON input rejects incomplete built-in elements: for example, `free_space` without `length_m`, `thin_lens` without a nonzero focal length, `thick_lens` without index/thickness, `interface` without `n_after`, `loss` without transmission, or a dispersion placeholder that is not an object. Incomplete table entries therefore cannot silently become inactive elements.

Known optional numeric fields set to JSON `null` are interpreted as unset during configuration normalization instead of reaching ABCD, FFT, random-ray, or preview code as `float(None)`. `phase_screen.power_transmission` is applied as power transmission in ABCD, FFT, and random Pump-ray paths. Mirror flips update the centroid, angles, and ellipse rotation together, avoiding inconsistent orientations of noncircular beams between backends.

`distance_before_m` is the actual free-space distance before an element. In the first row it is the distance from this beam's LASER source to its first element; in subsequent rows it is the distance from the previous element to the current one. In the common LASER → lens → GAIN setup, the GAIN row therefore gives the lens-to-crystal-entrance distance. Pump and Seed tables are stored independently and do not overwrite one another. Editing this column immediately updates the preview. **Shared crystal entrance = 0** is only the common reference plane used for overlaying beams and querying coordinates; it does not require equal source-to-crystal or lens-to-crystal distances. `turn_angle_x/y_deg` is a UI convenience field that inserts a coordinate break after the element.

The table and lower element editor apply changes differently. Table edits are written back immediately. In **Select Element to Edit**, the fields below the selector are a pending edit: click **Apply Selected Parameters Below** to write them to the path. For a `gain_medium`, the distance field is **Previous Element → GAIN Input Face Distance (m)**. Section 2.4 identifies the Pump and Seed GAIN rows in the bundled example.

`double_pass_lens_relay` is specifically for an ideal crystal/object plane → F3 → F4 → plane mirror → F4 → F3 → crystal/image plane system. The program derives the full round-trip ABCD matrix from the five actual distances/focal lengths. It accepts only unit-magnification imaging with `B=0` and `|A|=1`, then applies the exact image flip if `A=−1` and the return wavefront curvature `C/A` at the original sampled plane. This preserves the actual F3/F4 compensation power for the thermal lens while avoiding aliasing on a fixed FFT grid near intermediate foci. Systems with finite apertures, lens aberrations, or nonimaging `B≠0` must still be split into ordinary elements and checked for window/sampling convergence.

10.5 Spectral response

Mirrors, lens losses, and interfaces can define `spectral_response`:

```
{
  "type": "mirror",
  "power_reflectivity": 0.99,
  "spectral_response": {
    "wavelength_m": [2.2e-6, 2.4e-6, 2.6e-6],
    "power_reflectivity": [0.92, 0.995, 0.90],
    "out_of_range": "error"
  }
}
```

Wavelength-resolved calculations interpolate the response at each wavelength. `out_of_range` must be `error`, `edge`, or `zero`. The default is `error` so that unmeasured bands are not silently assigned endpoint coating performance.

10.6 Current limits of the optical bench

This is sequential paraxial propagation, not nonsequential CAD ray tracing. It does not automatically check for mount collisions, large-angle mechanical obstruction, ghosts, multiple reflection branches, or actual three-dimensional mirror placement.

A folded layout is unfolded along accumulated optical path, describing the order in which the field encounters components; it is not the experimental bench's global XYZ CAD geometry.

The backend also determines which elements are actually executed:

- Seed `optical_propagation="off"` ignores elements before the gain medium.

- Pump `prescribed_caustic` ignores source-to-gain elements.
- Pre-gain apertures, phase screens, and lens clear apertures in ABCD, `super_gaussian_abcd`, and hybrid propagation can only be represented by reduced/scalar approximations, not full spatial reshaping.
- Elements after the Pump `gain_medium` are not currently executed.

These combinations are recorded under `physics_validity.propagation_applicability`. Exploratory mode reports `review_required_outside_propagation_contract`; strict mode blocks quantitative validity. If an element matters to the conclusion, use an FFT/random-ray backend that actually supports it or revise the optical path rather than ignoring the warning.

---

11. Heat, stress, dispersion, and nonlinear effects

11.1 Pump heat-source models

The fast component ledger uses `heat_source_model="pump_component_ledger"`:

$$Q = \eta_{active} Q_{active} + Q_{passive} + Q_{ASE,passive} + Q_{Seed,enabled}.$$

Active Pump absorption is converted to heat using the quantum-defect model or an explicit fraction; passive Pump absorption becomes heat in full. Unabsorbed Pump, aperture losses, and lateral escape are not included in the crystal heat source. The separate terms keep active absorption, passive absorption, and optical-path losses from being assigned one common heat fraction.

For custom level systems or transition-by-transition closure, use `event_energy_ledger`. If the Pump-transition energy difference in custom `level_energies_J` disagrees with the Pump photon energy, the component ledger is marked inapplicable. Select the event ledger and declare the complete energy partition instead.

The event ledger explicitly assigns the energy of each decay and nonlinear reaction to four destinations: `heat`, `fluorescence_escape`, `ase_source`, and `unresolved`. Every fraction must lie in `[0,1]`, and the fractions for one event must sum to 1. With `require_complete_partitions=true`, a missing partition for any executed event causes an error; undeclared energy is not automatically treated as heat. Duplicate definitions of the same event by both name and index are also rejected before calculation, preventing a partition from being overwritten. A custom network must supply `level_energies_J` with one entry per expanded level.

Transient Pump, periodic waveforms, and `pulse_map` use linear end-of-step event counts consistent with backward Euler and accumulate the actual reactions executed in nonlinear substeps. They do not additionally apply trapezoidal integration to the event rates at the beginning and end of a step. Steady-state and `cycle_average` ledgers are explicitly marked as operating-point snapshots. `thermal.event_energy_ledger` reports power at each destination, changes in stored level energy, and pointwise/global residuals. Seed exchange, ASE boundaries, the spectral-diffusion bath, and interpulse dark relaxation belong to this identity only when their events have actually been registered. Strict mode does not present a local event ledger as full-system closure.

`finite_step_level_storage_derivative_W` independently evaluates stored-energy changes from populations before and after advancement. `finite_step_kinetic_storage_residual_W` checks finite-step kinetic events against that actual storage change; this local residual is not supplied when external interpulse Signal exchange is present. `pump_transport_kinetic_mismatch_W` separately reports the difference between optical propagation using bin-initial populations and implicit kinetic absorption. Time-step convergence is still required: a zero algebraic event-rate identity does not prove that Pump optics and populations are fully coupled consistently.

11.2 Quantum-defect heat fraction

When independent radiative-lifetime data and a measured total lifetime are available, `heat_fraction="lifetime_quantum_defect"` is recommended. Active Pump absorption first records the immediate energy difference between Pump photons and excitation of the laser upper level:

$$Q_{QD} = P_{abs,active}\left(1 - \eta_{exc}\frac{\lambda_p}{\lambda_{ref}}\right).$$

Upper-level nonradiative decay is then evaluated voxel by voxel as

$$Q_{nr} = N_{upper} h\nu_{ref} \left(\frac{1}{\tau_{total}(T, C)} - \frac{1}{\tau_{rad}} \right).$$

Because this expression uses actual populations, excitations already extracted by stimulated Seed/ASE emission are not counted again as nonradiative heat. This mode requires `energy_levels.upper_state_radiative_lifetime_s` and enforces `extra_heat_fraction=0` to avoid double counting the same quenching. If `spectroscopic_response` is enabled, τ_{total} automatically uses the operating-point or local three-dimensional temperature/concentration response. Quantitative evaluation is supported only for steady-state or cycle-averaged populations with `steady_state` or `periodic_steady_state.method="cycle_average"`. Exact `pulse_map` returns period-boundary populations rather than period-averaged populations, so that combination is rejected during configuration validation. Use the event-energy ledger when transient, event-resolved heat is needed.

For `heat_fraction="quantum_defect"`:

$$\eta_h = 1 - \eta_q \frac{\lambda_p}{\lambda_{ref}} + \eta_{extra}.$$

- η_q: Pump excitation quantum efficiency.
- λ_{ref}: the enabled effective fluorescence wavelength, or the Seed wavelength for legacy compatibility.
- `extra_heat_fraction`: an additional nonradiative heat fraction.

This is an effective heat ledger, not independent transport of fluorescence photons. If the fluorescence wavelength or quantum efficiency is not reliable, use a measured heat fraction and record its source.

For `heat_fraction="scaled_quantum_defect"`, the scalable form is

$$\eta_h = s\left(1 - \frac{\lambda_p}{\lambda_{ref}}\right) + \eta_{extra},$$

where `s=stokes_shift_multiplier`. This mode intentionally does not multiply by `pump_quantum_efficiency` again. Quantum efficiency still controls only the fraction of active absorption creating the target excitation. Do not add the two heat formulas together. Broadband Pump absorption and populations are calculated at each wavelength, but these two simplified heat fractions still use the Pump center wavelength; the results page explicitly reports this limitation.

For a broadband Pump, choose `spectral_quantum_defect` or `spectral_scaled_quantum_defect` instead. They do not represent the entire spectrum by its center wavelength. At each voxel they form normalized weights

$$w_j(x, y, z) \propto P_j \ \max\left[\sigma_{abs,j} N_l - \sigma_{em,j} N_u, 0\right],$$

then average the quantum-defect or Stokes heat fraction over wavelength with those weights. The result depends on local populations, temperature/doping-dependent cross sections, and the Pump input spectrum. It applies only to active Pump absorption; passive absorption remains 100% heat in the component ledger. All spectral heat fractions apply only to the active component. These are still not exact transition-event-rate heat models and do not include side-pumping/flashlamp ray tracing.

With the default `pump.spectrum.spatial_transport="shared"`, these two broadband heat modes do not independently propagate a spatial complex field for each Pump wavelength. They start from the total active-absorption power density already propagated and entered in the conservation ledger, then distribute it over wavelength using input spectral weights and each voxel's net active absorption, $\max[\sigma_{abs,j} N_l - \sigma_{em,j} N_u, 0]$. They capture spectral cross-section and local-population effects on heat fractions, but not different spot sizes caused by chromatic propagation. If that effect matters, select `spatial_transport="per_wavelength"`. Each sampled wavelength independently executes the chosen Pump optical train, diffraction, interfaces, apertures, and local absorption, returning `spectral_intensity_W_m2[lambda,z,y,x]` and wavelength-resolved active/passive absorption. The outer rate equations sum photon rates from all wavelengths in each voxel, retaining one shared physical population state. `shared` remains the fast reduced option.

11.3 Thermal-field models

- `off`: heat sources may be reported, but temperature remains at the reference value and thermal lensing is inactive.
- `steady_3d`: solves `div(k grad T)+Q=0`, supporting separate conductivities along all three axes.
- `transient_3d`: solves `rho*cp*dT/dt=div(k grad T)+Q` using implicit Euler.

`material.thermal.conductivity_W_mK` accepts either a positive scalar for isotropic conductivity or three positive finite values `[kx, ky, kz]`. Booleans, zero, negative, and nonfinite values are invalid. Conductivities in `thermal.entities` obey the same requirements.

The steady-state `auto` linear solver prefers SciPy sparse CG, falling back to SOR if it fails or SciPy is unavailable. Transient calculations require material density, specific heat, total duration, and a thermal time-step count.

11.4 Cooling boundaries

Each of the six faces can independently use:

- `dirichlet`: fixed temperature.
- `adiabatic`: zero normal heat flux.
- `water` / Robin: `-k*dT/dn=h(T-Tcoolant)`.

The water-cooling model represents only an equivalent heat-transfer coefficient on the solid side; it does not solve water velocity, turbulence, or pressure drop. Crystal, solder/foil layers, and a heat sink can be represented as discrete entities using `thermal.entities`, with fixed face contact resistance supplied by `thermal.contacts[].thermal_contact_resistance_m2K_W`. Contact pressure, friction, debonding, and temperature-dependent contact coefficients are not solved. Entities are assigned by regular-grid cell centers. Resolve thin layers with several cells and check grid convergence. This is multientity heat conduction, not multientity thermomechanical FEM involving a mount, solder, and foil.

Neighboring entities use the series face resistance `R_face=Δn_a/(2k_a)+R_contact+Δn_b/(2k_b)`, with a common face power on both sides. Entities must not overlap, their union must cover the crystal, and declared contact pairs must actually share cell faces on the grid. `thermal_contact_diagnostics` reports contact area, absolute heat-transfer power, and the mean temperature jump caused by contact resistance.

11.5 Thermal refractive index and stress

With `thermal_3d` enabled:

$$\Delta n = \frac{dn}{dT}(T - T_{ref}) + \Delta n_{stress}.$$

This three-dimensional field enters the corresponding Pump/Seed FFT backends as a phase screen at each z layer, so thermal lensing actually modifies their propagation. Random Pump rays with thermal refraction enabled use transverse `delta_n` gradients to update their direction. Reduced backends such as Gaussian/ABCD follow their own explicit applicability definitions. The mere presence of a three-dimensional `delta_n` result does not mean every propagator applied the same phase screen.

Stress modes are:

- `none`: no stress calculation.
- `isotropic_proxy`: a fast risk proxy based on temperature, E, Poisson's ratio, and thermal expansion. A scalar photoelastic coefficient can convert it to `Δn_stress`.
- `thermoelastic_fem`: three-dimensional isotropic, small-strain thermoelastic finite elements, off by default. Each active thermal cell corresponds to an eight-node hexahedral element. The solver satisfies `∇ · σ=0` and returns three displacement components, six stress and strain components, von Mises stress, the axial displacement range, and a discrete force-equilibrium residual. External surfaces are traction-free by default. `thermoelastic_fem.fixed_faces` can fully fix all three displacement components on selected crystal faces.
- `anisotropic_thermoelastic_fem`: uses a 6×6 elastic-stiffness matrix, six-component thermal expansion, and material-to-laboratory orientation, rotates them, and solves the same three-dimensional force

equilibrium. With `photoelastic_coupling="tensor_photoelastic"`, it also computes $\Delta B = p : \varepsilon$, constructs a transverse refractive-index tensor, and directly drives the monochromatic Jones field.

Anisotropic inputs consistently use Voigt ordering `xx, yy, zz, xy, yz, xz`. Shear components of stress and impermeability are tensor components; strain shear inputs are engineering shear, $2\varepsilon_{xy}, 2\varepsilon_{yz}, 2\varepsilon_{xz}$. Orientation can be supplied as an orthogonal 3×3 material-to-laboratory rotation matrix or Euler xyz angles, composed as $R_z \cdot R_y \cdot R_x$. Set the machine-readable declaration to `voigt_shear_convention="engineering_strain_tensor_stress"`. Convert data from other database conventions before use.

`thermoelastic_fem.maximum_active_elements` is an explicit computation budget. Exceeding it raises an error rather than silently coarsening stress hotspots. For fine optical resolution, retain a coarser physical/mechanical grid and enable an independent optical grid. FEM `photoelastic_coupling` defaults to `off` so that coefficients left in an old configuration do not automatically change the optics. Isotropic FEM can explicitly select `scalar_von_mises_proxy`; anisotropic FEM can explicitly select `tensor_photoelastic`. `endface_deformation.model="distributed_opd"` retains the separate front/rear FEM surface sags and applies boundary phase according to wavelength, propagation direction, and Pump/Seed role. It no longer folds the sag difference into the volume `delta_n`. This closure still does not resolve refraction by the deformed surface's local normal. Contacts, coatings/solder, heat sinks, and fracture statistics require dedicated models and experimental strength data.

If only the radial/tangential plane-stress approximation for an end-pumped rod is needed, the reduced Jones model in Section 9.9 remains available without FEM. Nonaxisymmetric temperature fields and crystal-orientation effects can use anisotropic FEM with tensor-photoelastic Jones propagation. Neither approach calculates a fracture safety margin.

11.6 Parabolic-fit diagnostics for three-dimensional fields

This diagnostic is off by default. When enabled, it fits area-weighted quadratic forms to the three-dimensional $\Delta n(x,y,z)$ and gain fields at selected z slices, reporting x/y and principal-axis equivalent focal lengths, fitting radii, orientation, and normalized RMSE. Select the fitting region automatically from the active material, specify it manually, or scale its radii from the Seed radius at the crystal entrance.

The module only compresses the current FVM/FFT result into an interpretable paraxial-curvature diagnostic. The fitted lens is **not** applied to the field as an additional ABCD element. Because three-dimensional Δn already enters FFT/BPM layer by layer, propagating through the same parabolic fit again would double count the thermal lens. Large RMSE means the field is not well represented by a parabola; use the full three-dimensional result in that case.

11.7 Local temperature/doping-dependent spectroscopic feedback

Available modes are:

- `off`: retain imported/manually specified cross sections and lifetimes.
- `operating_point`: correct them using one operating temperature and the three-dimensional density.
- `local_3d`: recalculate them from voxel temperature and density at every thermal-optical iteration.

Cross-section responses can independently affect Pump absorption, Pump emission, Signal absorption, and Signal emission. Supported models are `linear_scale`, `scale_table`, `boltzmann_stark_fraction`, and `absolute_temperature_wavelength_table`. The Stark model uses the material record's levels, degeneracies, and transition indices, normalized to its reference temperature.

Absolute cross-section tables replace reference cross sections while preserving explicit channel `absorption_scale` and `emission_scale` values, including zero. If the same Pump/Signal role also declares channel line shapes or detuning groups, a global absolute table does not define their individual responses, and that combination is rejected. Use explicit channel spectra and relative responses to avoid removing detuning information or multiplying in a measured spectral shape twice.

Lifetime temperature dependence supports linear and tabulated responses. Concentration dependence also supports Stern-Volmer:

$$\tau(N) = \tau_{ref} \frac{1 + (N_{ref}/N_q)^m}{1 + (N/N_q)^m}.$$

The default `range_policy="record"` clamps to endpoints outside the table and records the excursion in provenance and validity reports. `range_policy="error"` rejects it before propagation. The program does not supply missing material coefficients, and endpoint clamping is not an extrapolation model.

Materials with only `σ(T)` rather than a two-dimensional `σ(T,λ)` table are still evaluated at the selected operating temperature. Negative cross sections are rejected when applying material spectral data. Original negative values may be retained for inspecting the material record, but are not silently clipped to zero for propagation. Cross-section curves on the results page represent the **configured operating temperature**, not the full solved local three-dimensional temperature field. Preview points outside strict limits show gaps and a notice rather than a fabricated extension.

11.8 ASE and transverse parasitic lasing

`ase.model=six_ray` remains the fast ±x/±y/±z screening model. `angular_discrete_ordinates` uses paired, equally weighted Fibonacci-sphere directions to solve

$$\frac{dI_\lambda}{ds} = g_\lambda I_\lambda + j_{sp,\lambda}$$

along arbitrary directions.

Inputs include ASE wavelengths/weights, radiative lifetime, spontaneous-emission fraction, effective reflectivity on the six faces, boundary/population iteration settings, tolerances, relaxation, and a maximum-intensity safeguard.

By default, the interface retains separate `x_min/x_max/y_min/y_max/z_min/z_max` reflectivities. A pair shares values only when symmetric-face linking is explicitly selected. Boundary and population iterations check unrelaxed closure residuals; lowering the relaxation factor is not equivalent to convergence.

Outputs include three-dimensional ASE intensity, escaping power, passive-absorption heat, and a complete source/stimulated/absorbed/escaped energy ledger. Six-ray mode reports effective round-trip gain along each axis. The arbitrary-angle backend uses matrix-free power iteration to report the dominant parasitic eigenvalue of the reflective-boundary operator.

ASE is compatible with `steady_state`, periodic `cycle_average`, per-pulse `pulse_map`, and `reduced_pulse_map`. In a per-pulse mapping, the program solves quasi-steady ASE using populations after each Seed pulse, then holds that ASE field during the Pump/dark-time integration before the next Seed arrival. ASE is neglected during the short Seed pulse itself. The arbitrary-angle backend still resolves only external cuboid faces. Cylindrical/curved voxel boundaries do not have conformal specular reflection/TIR, and strict mode rejects treating them as resolved boundaries. This is an eigenvalue calculation for incoherent radiative-transport boundaries, not coherent wave-cavity modes.

11.9 Material dispersion

Materials may provide Sellmeier coefficients, GD/GDD/TOD/FOD per meter, and a reference wavelength. Sellmeier data determine wavelength-dependent index and material phase. Manually entered dispersion is added as a correction; avoid representing the same material contribution twice.

11.10 Kerr, SPM, and B-integral

In the coherent time domain:

$$\Delta\phi_{Kerr} = k_0 n_2 I \Delta z.$$

The default `seed.coherent.local_operator.scheme="symmetric_adaptive"` advances within each physical z cell using half diffraction → thermal phase → half dispersion → half nonlinearity → gain/loss → half nonlinear-

ity → half dispersion → half diffraction. It subdivides automatically according to `|g ds|`, the material-dispersion phase span, transverse thermal-phase PTV, Kerr phase, and strong-field loss. Legacy `scheme="sequentia l"` remains available for historical compatibility and fast weak-coupling calculations. These are local operator-change limits, not a complete discretization-error estimate. Exceeding a threshold at `maximum_substeps` is recorded, and a worst excursion is not overwritten by later normal steps. The result must pass the relevant validity gates and independently demonstrate spatial/temporal convergence. Material dispersion interleaved with strong-field effects is checked even when the strong-field option is enabled without Kerr.

The program reports the maximum phase in an individual substep and the B-integral accumulated at fixed transverse grid cells across all z layers and passes. Without transverse walk-off/diffraction, this is the path integral. With transverse transport, it is only an Eulerian diagnostic, not the Lagrangian B-integral of an actual ray. Kerr does not directly change energy or generate heat.

11.11 Multiphoton absorption and free carriers

Multiphoton absorption follows

$$\frac{dI}{ds} = -\beta_K I^K.$$

Here `s` is the actual beam path inside the crystal and `I` is intensity in a cross section perpendicular to that beam. At oblique incidence, crystal-face projected intensity or axial `dz` cannot be substituted directly. Every path step is integrated analytically, preserving positive intensity even under strong loss. The units of `beta_SI` depend on K, and its value must come from data for the relevant material, sample, wavelength, and pulse duration. For K>1 the analytic solution is `Iout=Iin/[1+(K−1)βK Iin^(K−1) ds]^(1/(K−1))`. Adaptive subdivision estimates loss using this nonlinear relation. For high-order KPA, simply dividing the whole-cell loss fraction between substeps is incorrect. A step-count limit controls computation cost; it does not guarantee accuracy.

Carrier density follows `dN/dt=G−N/τ`. FCA/refraction uses the average density within a bin rather than only its endpoint. Starting with zero carriers and `dt`≪τ, the mean density tends to `G dt/2`. This is the stable analytic limit; it does not include undeclared carrier diffusion, energy transport, or coherent multimode statistics. The current incoherent-HG nonlinear closure uses the summed intensity. If the experiment exhibits rapid random interference, ⟨`I^K`⟩ need not equal ⟨`I`⟩`^K`, and a separate statistical model is required.

`strong_field.mechanism` declares only the physical interpretation of this effective power law; it does not change the propagation equation:

- `host_interband`: host interband multiphoton absorption. Both `provenance.bandgap_eV` and a reference wavelength are required. The program checks that K is not below the bandgap threshold at that reference point.
- `measured_sample_effective`: an effective Kth-order loss measured on the same sample.
- `impurity_sequential` / `defect_assisted`: an effective power law for sequential impurity absorption or defect-assisted absorption.
- `phenomenological_sensitivity`: sensitivity scans only; strict mode does not allow quantitative mechanistic conclusions based on it.

The bandgap criterion answers only how many photons are minimally required for a pure-host interband transition. It does not rule out intermediate dopant levels, defect states, sequential absorption, or effective nonlinear loss from sample scattering. Conversely, an approximately I^2 roll-off alone does not prove host TPA. Record sample identity, measurement wavelength, pulse duration, β uncertainty, and the original data/reference. For broadband pulses, one K remains an effective model near the reference wavelength; check sensitivity to the spectrum and model form.

The form and JSON provenance definitions correspond as follows: `provenance.status=unverified|provisional_sensitivity|literature|sample_measured`; `reference` stores the original paper/data citation; `sample_identifier` identifies the measured sample; and `reference_wavelength_m`, `reference_pulse_duration_s`, and `relative_uncertainty` retain the β measurement conditions. Relative uncertainty is a fraction, for example `0.15` for 15%. `host_interband` additionally requires positive `bandgap_eV` and a reference wavelength to check minimum K. That check is not proof of the actual loss mechanism. When recording of β

measurement conditions is disabled, the last three optional fields are written as `null` so that old hidden values are not mistaken for current evidence. This cannot be disabled for `host_interband` because those fields are required by its physical definition.

The optional free-carrier model includes MPA generation efficiency, carrier lifetime, FCA cross section, and refractive-index change per unit carrier density.

`coherent_cpa` advances local carrier density on the x-y-t sampled field and may retain its three-dimensional state across pulses in strict `pulse_map` calculations. `commercial_app_fast + effective_area_spm + strong_field_effective_area` instead advances only an energy-equivalent scalar carrier state coupled to the one-dimensional temporal envelope, closing its cross-pulse steady state locally. Both paths report loss energy and peak carrier density, but Commercial App values cannot be interpreted as radial carrier distributions or full spatiotemporal coupling.

Both paths use `deposit_fraction_as_heat` to determine the fraction of irreversible loss entering the thermal field. This is an effective net thermalization fraction on a cycle-averaged scale, not an explicit internal carrier-energy state. The energy ledger accordingly retains `full_first_law_closed=false`. When carrier storage, radiative recombination, or transport is significant on thermal timescales, an independent carrier-energy model is needed; the fraction cannot be interpreted as an instantaneous thermalization rate.

---

12. Numerical grids

12.1 Main physical grid

`grid.size_x_m/size_y_m/length_z_m` and `grid.center_x_m/center_y_m` define the physical computation box in fixed laboratory coordinates. Nx/Ny/Nz are the numbers of voxel centers. The main grid stores populations, doping, Pump absorption, heat sources/temperature, stress, and mapped-back Seed extraction/loss. Moving the physical grid does not automatically move the crystal, Pump, Seed, or optical windows; they retain their own absolute coordinates.

Nz subdivides only the crystal length. Air paths outside the crystal are propagated through ABCD or external FFT elements, not through main-grid z cells.

12.2 x/y grid types

- `uniform`: uniform sampling, directly suitable for FFT.
- `center_stretched`: a sinh mapping concentrates cells around `grid.refinement_center_x_m/refinement_center_y_m`, with progressively larger cells near the edges. If an axis is `null`, its refinement center follows `grid.center_x_m/center_y_m`, retaining symmetric central refinement for older configurations. An explicit center refines an offset spot without moving the computation box. The refinement center must be strictly inside the corresponding physical window.
- `refined_region`: a uniformly refined core with dimensions `grid.refined_region.size_x_m/size_y_m` and cell counts `grid.refined_region.nx/ny`. Outer cells grow geometrically while the physical box boundaries remain fixed. The core follows `refinement_center_*_m` and must lie strictly inside the box. At least two outer cells are required on each side; invalid dimensions are not clipped to force a grid to fit.
- `stretch_x/y`, used for `center_stretched`, is the continuous mapping's local scale ratio between the edge and the refinement point. Larger values produce smaller cells near the refinement point. For finite Nx/Ny, the actual largest/smallest cell-width ratio is slightly below this continuous limit.

Independent refinement centers change only the distribution of physical cells inside fixed boundaries. They do not change the crystal mask, beam centers, boundary conditions, or Pump/Seed optical windows. For a beam that is later offset, generally set `refinement_center_*_m` to the laboratory coordinates of the region of interest. Whether the crystal should move is a question of actual alignment geometry, not a way to trigger grid refinement.

Integration, thermal finite volumes, and thermoelastic FEM on nonuniform grids use actual cell boundaries. A nonuniform grid cannot directly support a standard FFT, so enable an independent uniform optical grid or use a reduced combination that does not require FFT.

Each main-grid axis requires at least 3 cells. The form does not clip valid imported grids to fixed display limits such as 256/300, and Pump temporal sampling is not capped at 500. Memory, FEM element budgets, and each model's sampling requirements still limit the calculation.

12.3 Independent optical grids

`grid.optical.enabled=true` defines a local uniform Seed/BPM grid. By default, the Pump shares it through `grid.pump_optical.mode="shared"`. When a broad Pump and a narrow Seed/BPM require different windows, choose:

- `physical`: the Pump directly uses the physical grid. Locally refined grids are suitable only for Pump backends that do not require a uniform FFT.
- `custom`: an independent uniform Pump grid defined by `grid.pump_optical.size_x_m/size_y_m/nx/ny/center_*`.

Populations and thermal fields always remain on the full physical grid. Cross-grid mapping distinguishes continuous state variables, conservation ledgers, and inventories with memory:

- Intensive continuous quantities such as populations, densities, temperature, and Δn are mapped to optical grids using bounded bilinear interpolation of cell-center values. This preserves constants, reconstructs linear fields exactly, and avoids phase steps at cell boundaries.
- Pump absorption, Seed extraction/loss, and population changes are mapped back to the physical grid by finite-volume cell overlap, preserving transverse integrals and ledger conservation.
- The free-carrier input state is a special inventory with memory. Forward mapping uses smooth interpolation, followed by particle-number normalization over the actual crystal support in each z layer. After propagation, changes on the optical grid are conservatively mapped back, then projected to a nonnegative state with the target total particle count. Results report the initial remapping correction, write-back correction, and total-carrier-number change error.
- Pump, Seed, and physical grids currently share Nz, but may have independent x/y windows and sample counts.

Do not reverse the conservative write-back operator and use it for interpolating a continuous phase field. A piecewise-constant Δn can integrate correctly while introducing nonphysical high spatial frequencies at physical cell boundaries and substantially increasing M^2. `ledger_remap_method` and `physical_to_optical_intensive_method` in the summary record these two distinct mapping definitions.

Each saved optical plane retains its own x/y coordinates, so an 80×80 Pump and 128×128 Seed can be displayed and exported correctly within one result. `summary.grid_coupling.pump/seed` report their mapping errors separately, and `shared_optical_grid` states whether they share a grid. The same summary records the physical-grid center, resolved refinement centers, and whether each axis was explicitly specified, supporting result traceability.

12.4 Choosing the FFT window

The window should span at least several times the largest beam radius, with nearly zero field at the edges. Check the entrance sampled fraction. FFT uses periodic boundaries, so a window that is too small can cause clipping or wraparound, appearing as an abnormal output drop, focal-spot deformation, or a sudden change in a thermal-optical fixed point. Results also check energy in the outer 5% of every recorded optical plane. If it exceeds 1%, that plane's profile and sampled M^2 are explicitly flagged for finite-window risk.

`hybrid_abcd_fft` can retain the full Gaussian power outside the crystal and use a local window only at the crystal entrance. This avoids sacrificing in-crystal resolution to cover a large source plane.

12.5 z resolution

Nz affects Pump depletion, Seed split-step propagation, axial heat-source distribution, JOSA gain steps, Kerr, and strong-field loss. The coherent symmetric adaptive operator subdivides optical propagation within physical z cells, including the corresponding diffraction/phase operations. These substeps do not increase the spatial cell count of thermal fields, material properties, or populations on the physical grid, so Nz convergence must still be

checked. Inspect `max |gΔz|`, maximum nonlinear phase per step, loss per step, actual local substep counts, and whether the substep limit is reached.

---

13. Parameter sweeps

Parameter sweeps answer how outputs change when an input is varied; they do not use a separate physical solver. At each sample, the program copies the full current configuration, changes only the swept field, and reruns the same `AmplifierSimulator`. Supported optical inputs can inherit the preceding point's state as an initial guess.

13.1 Inputs available for direct sweeps

The interface's supported list includes:

- Pump power.
- Pump/Seed center wavelengths. If an absolute-wavelength file is used, sweeping the center wavelength does not translate the file's entire wavelength axis.
- Pump/Seed distances from LASER source to first element.
- Pump/Seed distances from the preceding element to the first GAIN. In the common LASER → lens → GAIN path, these are the individual lens-to-crystal distances. Pump and Seed are adjusted independently. Subsequent Seed hits and interpass propagation are managed by `simulation.multipass`.
- Common scaling of the x/y waist radii of Pump or Seed.
- Seed repetition rate, single-pulse energy, and individual x/y waist radii.
- Individual Pump x/y waist radii.
- Crystal length, refractive index, and ion density.
- Pump absorption cross section, Signal emission cross section, and upper-state lifetime.
- The number of crystal passes.

Select linear or logarithmic sampling, start, end, and point count. Logarithmic sampling requires positive start and end values. Enabling the **two-dimensional parameter grid** allows a second, different input. The program calculates the Cartesian product and displays each output metric as a two-dimensional heatmap. A 2D sweep requires Nx×Ny calculations, not Nx+Ny. For detailed FFT/thermal-fixed-point models, start with 3×3 or 5×5 to inspect the trend.

Choose the outputs to plot, then click **Run Parameter Sweep**. Where points can run independently, **Concurrent Points** selects 1-3 workers. One worker provides the full live diagnostic panel; 2-3 workers show each point's stage and progress while still saving complete fields per point. Supported warm-start sweeps run serially so that each point can inherit the preceding state.

Pause Sweep freezes the active point or points and the sweep process; **Resume Sweep** continues from the same numerical state without repeating completed points. **Stop and Cancel This Sweep** terminates unfinished points but retains completed points for download. **Download Completed Portion** includes only completed, persisted points, not the active or paused points; it becomes available after the first point finishes. Save the downloaded archive before choosing **Discard This Sweep**, which permanently deletes the temporary results. Pause/resume has the same platform and server-lifetime limits as single-run controls in Section 14.1.

FCA convergence stopping-condition switch

`simulation.periodic_steady_state.free_carrier_convergence.enabled` defaults to `true`. Setting it explicitly to `false` removes only the carrier-residual stopping restriction from the ordinary inner iteration, Joint Frozen joint criterion, and final frozen check. FCA absorption, carrier evolution, phase terms, and raw residuals are still calculated and retained. The interface option for using the FCA carrier residual in convergence decisions controls this setting. Population, output, and thermal-optical tolerances remain active, and original validation checks are not erased. Meeting the remaining stopping conditions does not mean the carrier state has converged.

Warm starts and 2D traversal order

Automatic warm starts are supported for Pump power, individual x/y waist radii and common scaling of Pump/Seed, LASER-to-first-element distances, and preceding-element-to-first-GAIN distances. The first point uses the configured initialization or a user-selected archived initial state. Later points inherit populations, temperature, and thermal-optical state from the most recent converged point. Every point still propagates again and executes the original convergence checks; old output is not reused.

When both 2D axes are supported, automatic serial execution follows a serpentine order: the first row runs left to right, the second right to left, and so on. Starting each new row from its neighboring state avoids a jump across the entire parameter range. Plots and archives retain the original X/Y coordinates. Every point's parameters are generated from the sweep baseline; waist scaling is not multiplied cumulatively between points.

Failed or explicitly unconverged points do not export a new continuation state. A previous usable state is retained if available. Incompatible state shapes or values are recorded, and the calculation falls back to configured initialization. `continuation` records whether a state was inherited, its source point, and any fallback reason. The old `pump_continuation` field remains as a compatibility alias. Numerical convergence and overall validation are assessed separately; all original warnings are retained.

Two-dimensional combinations containing unsupported parameters, such as crystal dimensions or material properties, still initialize every point independently and can run in parallel. Lightweight `run_compact_parameter_*` screening interfaces also retain independent-point calculations. Warm starts may follow one stable solution branch. For multistability or hysteresis, compare reverse scans or independent initial conditions; faster convergence does not establish uniqueness.

Starting from an archived point

Open **Local Result-Point Warm Start (Optional)**, enter the full local path to a complete `.ampres.zip`, and enter its **Result-Point Number (Starting at 1)**. Leave the path blank to disable this option. It supplies an initial guess for a new single run or the first point of a supported warm-start sweep, rather than reusing old output or resuming an interrupted job. Sweeps that do not support state continuation, including a crystal-length sweep, use the configured initialization and ignore this archive even with one worker. The archived energy-level names and physical-grid coordinates must match the current setup; incompatible inputs are rejected. The solver still closes every equation, including any newly enabled free-carrier state.

13.2 Outputs available for plotting

Available metrics include output pulse energy, output average power, total energy gain, output Seed M^2 x/y, absorbed Pump power/fraction, the post-pump fraction in the main Signal upper level, inversion of the main Signal transition after pumping, total thermal load, peak temperature, net Seed extraction energy, and maximum local gain.

Run validity and individual metric validity are retained separately. For example, output power can remain valid while an M^2 diagnostic is invalid because of a finite FFT window. That point can appear in the power curve but not the M^2 curve. Gain/heat extrema are computed only inside the crystal mask. Mean population fractions and inversion use actual volume weighting over positively doped crystal cells. If an optional backend safely falls back to a reduced model, each point records both requested and executed Signal models. One-dimensional curves are grouped by the executed backend, and two-dimensional heatmaps switch by the executed backend, so a model change is not presented as a continuous physical trend.

Linear/logarithmic plot coordinates follow sweep settings. Invalid or missing metrics and 2D cells belonging to other backends retain their full coordinates but are left blank. The exception for displaying raw live 2D values is described in Section 3. Failed points retain error messages; missing points must not be interpreted as zero. For expensive modes, establish the trend with 3-5 points before increasing the count.

13.3 Interpreting roll-off curves

An output that flattens or falls as Pump power increases can arise from gain saturation, spatial Pump/Seed mismatch, reduced overlap caused by thermal lensing, temperature-dependent cross sections/lifetimes, ASE/TPL, passive Seed absorption, aperture/FFT-window clipping, nonlinear absorption, or related effects. Toggle mech-

anisms individually and inspect the energy ledger and warnings together. One output curve alone does not identify the physical mechanism.

13.4 Monte Carlo uncertainty and experimental calibration

Use an ordinary sweep to examine trends over a prescribed grid and the uncertainty workflow to propagate batch variation and measurement uncertainty. A study file can assign a `normal`, `lognormal`, `uniform`, or `triangular` distribution to each configuration path and describe correlations with a correlation matrix.

After installing the package and activating its environment, an illustrative command using the bundled baseline is:

```
amplifier-sim uncertainty configs/example.json uncertainty-study.json -o
 ↪  output/uncertainty.json
```

`uncertainty-study.json` is a user-supplied study definition, not a file bundled with this release. Supply distributions, parameter paths, and correlations appropriate to your measurements.

Only samples satisfying the shared `summary.validation.valid=true` criterion contribute to means, standard deviations, and quantiles. Failed or strictly invalid samples retain their parameters, errors, and validity reports and are never inserted into statistics as zeros. Each metric also independently filters unavailable diagnostics. Different executed Seed backends are summarized separately; mixed backends do not produce a combined `metric_statistics` object.

Experimental calibration uses observations, measurement standard deviations, and optional `model_error_standard_deviation` in weighted nonlinear least squares. Parameters may have bounds and Gaussian priors:

```
amplifier-sim calibrate configs/example.json calibration-study.json -o output/calibration.json
```

`calibration-study.json` is likewise a user-supplied study definition and is not included in this release.

Outputs include the best parameters, standardized residuals, χ^2, all evaluation records, and the local Jacobian-based parameter covariance. Covariance is obtained from the Jacobian's SVD. If the Jacobian is rank deficient, covariance is `null` rather than a zero variance that falsely suggests the parameters are determined. The default study setting `residual_variance_mode="known_absolute"` uses the known measurement/model-error variances without multiplying by reduced χ^2. Only `estimated_from_residuals` estimates an additional scale from residuals; insufficient degrees of freedom make covariance unavailable. Inspect `parameter_covariance_status`, `parameter_jacobian_rank`, and the actual `executed_signal_model`.

Optimization and final reevaluation must use the same executed Seed backend. A fallback that changes the backend causes rejection of further fitting. The fit does not provide a complete Bayesian posterior. For multimodality, strong nonlinearity, or structural model error, use additional MCMC/surrogate-model cross-checks.

---

14. Running calculations and viewing results

14.1 Checks before running

Check the following in order:

1. Units, especially μm/mm, cm^{-3}/m^{-3}, mJ/J, and MHz/Hz.
2. Whether the Pump and Seed paths hit the same crystal.
3. Whether a D4σ radius has accidentally been treated as a diameter.
4. Whether material cross sections match the selected wavelength, temperature, and polarization.
5. Whether the FFT entrance sampled fraction and edge intensity are sufficiently safe.
6. Whether the effective model matches the interface selection.
7. Whether the steady-state method, repetition rate, and Pump period are consistent.

The interface saves the resolved configuration and `SimulationResult` for each calculation. Later edits to the form do not recalculate that result, so interpret it using its saved configuration.

Starting, pausing, and canceling

After the configuration checks pass, click **Run Current Configuration**. The background worker uses a fixed configuration snapshot. Editing the form, changing pages, or generating a configuration file does not modify or restart the active calculation; these edits apply to the next run.

Pause Solve freezes the process and preserves its in-memory numerical state. **Resume Solve** continues from that same state. **Stop and Cancel This Solve** terminates the job, and an unfinished result is not archived. Pause/resume requires process-signal support, as provided on macOS/Linux; the current implementation does not support it on Windows.

Keep the Streamlit server and the terminal that launched it running. Closing or refreshing the browser alone does not cancel a worker, but stopping the server does not save a checkpoint. An unfinished calculation cannot be resumed from its previous numerical state after a server restart. Reopening a completed `.ampres.zip` restores saved results for viewing, not a running solver.

A completed task means execution finished. Check convergence, validation, and warnings before using the result quantitatively. After a failed or canceled run, an earlier completed result may remain visible; check its configuration snapshot and the failure or cancellation notice before interpreting it.

14.2 Overview metrics

The overview highlights the final system output energy per pulse. If the repetition rate is positive, it also calculates

$$P_{out,avg} = E_{out,pulse} f_{rep}.$$

The displayed output power is the reported pulse energy multiplied by the Seed repetition rate; it is not an instantaneous power from one period merely relabeled in W. Interpreting it as a steady-state mean requires convergence of the appropriate periodic operating state. Absorbed Pump power is the volume-integrated absorption averaged over the corresponding steady-state or sampling window, in W. A single-shot preparation window must not be described as a repetition period; see Section 8.6.

14.3 Optical paths and passes

The optical-path page displays Pump/Seed envelopes, elements, the shared crystal position, and energies by pass. In multipass systems, distinguish:

- Energy entering at the source plane.
- Energy immediately before/after every crystal hit.
- Active output after the gain medium.
- System output after lens, mirror, and output-coupling losses.

Do not substitute the final pass's active output for system output; passive losses may still lie between them.

Jones calculations also display Stokes parameters, integrated degree of polarization, and the orthogonal-analyzer fraction relative to the input state. `output_jones_field` in `fields.npz` retains complex `Ex/Ey`; ordinary optical-plane and Seed-pattern views still display the total intensity of both polarizations.

When complex-field planes are recorded, output Seed M² is also reported. `M2_x_sampled/M2_y_sampled` in the plane table are calculated entirely from the second moments of that plane's complex amplitude and therefore respond to field distortions caused by apertures, nonuniform gain, thermal phase, and diffraction. For a coherent source configured with `M²=1`, this is the scalar coherent-field diagnostic. For an effective or partially coherent source represented by a single scalar `M²>1`, the program does not have a full mutual-intensity matrix. Its headline result therefore uses the raw field-distortion factor multiplied by input transport M² and is explicitly labeled as an effective diagnostic. It neither presents unchanged input M² as an output measurement nor equals an experimental ISO 11146 multiplane caustic fit.

For direct comparison with an experimental caustic, enable `seed.caustic_diagnostics.enabled`. The program independently propagates the system output complex field to `positions_m`, or equally spaced planes between `start_m` and `stop_m`, calculates D4σ radii at every plane, and fits `w²=w0²+theta²(z−z0)²`. Results include caustic M², output-plane phase-space M², power drift, window-edge energy, and ISO-style near-waist/far-

field sampling coverage. The scan requires an available monochromatic output complex field. Spectrally collapsed broadband intensity or incoherent summation of causal time bins has no unique output phase. In those cases, complex-field caustic/M² diagnostics stop and explain why rather than fabricating a wavefront from the square root of intensity. Saving two-dimensional caustic spots also requires the global `record_optical_planes` switch.

When marking a requested scan start, end, or another anchor, the interface interpolates D4σ between neighboring calculated caustic planes. It does not present the nearest preview sample as the exact anchor. Formal fitting still uses only the scan planes actually solved.

In addition to quadratic fitting, `material.field_diagnostics` reports radial and x/y profiles of integrated OPD after removing quadratic terms. It also performs an area-weighted real Zernike decomposition over a selected elliptical aperture; `n=4,m=0` is primary spherical aberration. The fitting aperture changes the coefficients, so use the same aperture before comparing with a paper or wavefront sensor. For one pass, `OPD(x,y)=sum_z delta_n(x,y,z)*delta_z`. Piston, x/y tilt, x², y², and xy are removed jointly before fitting orthonormal real Zernike functions on the ellipse-normalized unit disk. Discrete clipping can destroy ideal orthogonality, so inspect the design-matrix condition number and full-fit residual rather than only one spherical-aberration coefficient.

Every caustic position is independently propagated from the same system output field. The fit uses `w²(z)=w0²+theta²(z−z0)²` and `M²=pi*n*w0*theta/lambda0`. The summary also retains output-plane phase-space M², scan power drift, energy in the outer 5% of the window, and near-waist/far-field coverage. This numerical diagnostic follows the ISO 11146 approach but does not include real camera background, dynamic range, pixel calibration, or measurement uncertainty.

14.4 Three-dimensional fields and two-dimensional slices

Available views include Pump intensity, Pump absorption, populations of individual levels, post-pump inversion, Seed fluence, gain, active extraction energy, passive/nonlinear-loss heat, total heat source, temperature, refractive-index change, and stress proxies.

Three-dimensional views should retain the actual crystal x:y:z proportions. Crystal-focused display compresses only long external optical paths. Select two-dimensional slice positions by physical coordinates and use each field's own coordinates and units.

In minimum/mean/maximum statistics for three-dimensional fields, means are weighted by actual cell volume. Cross-sectional means in axial profiles are weighted by x/y cell area. On a `center_stretched` nonuniform grid, these are not arithmetic means of array elements.

14.5 Spectral and temporal results

Wavelength-resolved and coherent modes display input/output spectra, gain narrowing, input/output temporal waveforms, spectral phase, and compressed temporal power. A `spectral_time` temporal curve represents ordered energy bins; a `coherent_cpa` temporal curve is transformed from the complex spectrum under `exp(−iωt)`. The two curves therefore require different physical interpretations.

Two-dimensional optical-plane color scales use actual fluence in `J/m²` or power density in `W/m²`. Only profiles explicitly labeled normalized are relative values. Time displays in ps and fs are converted separately; they must not share one conversion factor.

14.6 Diagnostics page

The diagnostics page collects:

- Configured values, effective values, and their provenance.
- Energy/power ledgers.
- Maximum Pump absorption optical depth per cell, `alpha*ds`, transverse escape inside the crystal, and power-closure residuals.
- FFT-window sampling and aliasing risks.
- JOSA CFL, fallback, and enforced-coupling audits.
- Pulse-duration/population-lifetime ratios and whether explicit relaxation covers long-pulse effects.
- Population clipping of negative values, renormalization magnitude, and worst sources.

- Thermal-optical fixed-point residuals and iteration history.
- Periodic steady-state population and output residuals.
- ASE thresholds, strong-field losses, and B-integral.
- Runtime architecture, FFT backend, and warnings.

Before drawing quantitative conclusions, resolve warnings about energy nonconservation, window clipping, non-convergence, parameter-range excursions, or automatic model reduction.

---

15. Steady states, periods, and time samples per period

15.1 Three common temporal questions

The program distinguishes three timescales:

1. Sampling the Pump waveform within one repetition period.
2. Temporal/frequency sampling inside one Seed pulse.
3. Population convergence over repeated Pump+Seed periods.

Each requires its own sampling or convergence check.

15.2 Samples per period with a CW Pump

A CW Pump has the same power at every sample. Changing 40 samples to 80 does not apply twice as much Pump or double the energy per period. It divides the same period into finer integration steps and advances populations with a smaller `dt`. Integration weights preserve the total Pump duration.

For CW pumping and smoothly varying rate equations, relatively few samples are often enough. Narrow Pump pulses, strong depletion, or rapid level transfer require finer sampling and convergence checks.

15.3 `cycle_average`, `reduced_pulse_map`, and `pulse_map`

- `cycle_average` inserts period-averaged Pump and Seed photon fluxes into algebraic steady-state equations. It is fast and suitable for high repetition rates with weak single-pulse extraction.
- `reduced_pulse_map` is a coherent-CPA initializer: a phase-insensitive or axisymmetric coherent proxy supplies an approximate periodic population, then the configured full `pulse_map` iterates to its own convergence criteria. The legacy method name selects this same initializer; the proxy is not a separate reported solution.
- `pulse_map` executes a complete Pump period and one Seed extraction pulse as a fixed-point map, repeating until both pre-pulse populations and output energy converge. It suits strong pulse extraction, significant front/tail depletion, or memory within the period. Cycle-averaged initialization, outer thermal-optical warm starts, and safeguarded Anderson acceleration are available by default to reduce map evaluations.

The final `pulse_map` output is one representative Seed pulse from the converged period. Average output remains this pulse energy multiplied by repetition rate.

Pump/population time samples per period form the rate-equation temporal grid, dividing one repetition period into population-advancement intervals. For a CW Pump, adding samples only reduces the population time step; it does not create additional Pump pulses. The thermal transient's FEA time-step count belongs to the temperature equation. The two grids serve different equations and generally need not use equal step counts.

15.4 Convergence criteria

The periodic fixed point checks both relative population and output residuals. If coherent cross-pulse free carriers are enabled, the selected combination of FCA optical depth/carrier phase/absorbed energy, or the full carrier-density field, provides a third stopping condition. The minimum iteration count guards against accidental early stopping; the maximum prevents the iteration from continuing indefinitely. If any gate remains above tolerance, even a generated result must be treated as an unconverged approximation.

---

16. Command Line, Batch Processing, and Reproducible Runs

Use the web interface to adjust parameters and the command line for batch processing, version control, and saving results for formal analysis. Before running the commands below, activate the environment installed in Section 2 and open a terminal in the extracted project folder.

16.1 Validate and Inspect the Merged Configuration

```
amplifier-sim validate configs/example.json
```

This command reads the JSON, merges it recursively with the defaults, checks the fields and physical settings, and prints the complete configuration passed to the solver. If values in the interface differ from those used in a calculation, inspect this output first.

16.2 Run and Save Results

```
amplifier-sim run configs/example.json -o output/example
```

Alternatively, run the installed package directly:

```
python -m amplifier_sim run configs/example.json -o output/example
```

16.3 List Built-in Models

```
amplifier-sim models
```

The output groups the available energy-level models, presets, Signal/JOSA models, temporal and spectral profiles, common optical elements, beams, and stress plugins. It lists their names rather than every configuration field. The specialized `double_pass_lens_relay` element, for example, is described in Section 10.4 and checked by the configuration validator.

16.4 Embed External Material Data

This command requires a material-library file in a supported format. Replace `supplied-material-library.json` with your own file; a separate material library is not included in this distribution.

```
amplifier-sim materialize configs/example.json supplied-material-library.json \
  --material 'Cr:ZnS' --doping-percent 0.72 --temperature-K 293.15 \
  --polarization x --snapshot-out-of-range edge -o configs/crzns_materialized.json
```

Choose the material name and operating point for your library and sample. The output embeds the operating-point cross sections, full response data, and source SHA-256 so that they can be kept with research or experimental records.

16.5 Convergence Studies

```
amplifier-sim convergence configs/example.json \
  --axes physical_nx physical_ny physical_nz optical_window optical_sampling \
  --factors 1 1.5 2 --relative-tolerance 0.01 \
  -o output/example_convergence.json
```

Each axis is refined separately from the same baseline configuration, and inapplicable axes are recorded as skipped. Refining the physical Nx/Ny or `fem_mesh` also increases `refined_region.nx/ny`, while preserving the core size and computational-box boundaries. The refinement therefore covers the beam core as well as the surrounding cells. Separate axes cover time and spectral sampling, the independent Pump window, coherent soft windows, ASE spectrum and directions, and ray count. Run `amplifier-sim convergence --help` for the complete list. Automatic certification requires at least three valid, distinct discretizations, with both of the last

two adjacent-level changes below the default tolerance of `0.01`. Certification is unavailable with only two levels, sampling counts that become identical after rounding, or no actual refinement. Even a certified study does not provide a global error bound relative to the continuous exact solution.

The report compares runs using their effective backend and discretization. Failed or invalid runs, invalid target metrics, or backend changes cannot be certified as converged; the corresponding `converged` is `null`, with a reason. A pass applies only to the evaluated axes and metrics; skipped items remain unverified. The CLI returns `3` when `all_assessed_axes_converged` is false. This overall flag treats `sensitivity_complete` for the random-seed axis as completion of a sensitivity check, rather than certification that random error has vanished. Inspect each axis's `interpretation` and status as well.

`pump_ray_seed_sensitivity` measures sensitivity to random seeds and must not be interpreted as grid convergence. `pump_ray_count` also runs three different seeds at each level. The 95% interval for the mean of repeated samples uses Student's t distribution with N−1 degrees of freedom, assuming independent repetitions with approximately normally distributed results. With only one sample, the standard deviation, standard error, and interval are all `null`; without SciPy, the interval is unavailable. The interval does not include discretization bias, material uncertainty, or model error. For the seed-sensitivity axis, `--factors` generates different seeds according to the implementation. Choose multiple distinct repetitions and inspect the actual seeds recorded in the report.

16.6 Uncertainty and Calibration Commands

Section 13.4 describes the study-file formats and statistical definitions for `uncertainty` and `calibrate`. `uncertainty` returns `4` when there are no valid samples. `calibrate` returns `5` when optimization terminates normally without succeeding. Configuration errors, missing required dependencies, and backend changes during calibration raise errors. Check the reported status rather than assuming that a written JSON file means the calculation succeeded.

16.7 Published Benchmarks

```
amplifier-sim benchmark
amplifier-sim benchmark sridharan_2007_ybyag
```

The first command lists benchmarks. The second runs the named benchmark, whose source is documented, and checks its result against the specified tolerance. This example evaluates FN numerical values from a published design prediction. It neither runs the current Setup's full three-dimensional coupled calculation nor establishes agreement with measured output. The output fields `claim_type` and `evidence_note` describe what the benchmark supports.

---

17. JSON Configuration Reference

17.1 Merge Rules

Configurations use JSON. You can specify only the fields to override; the loader merges them recursively with `DEFAULTS`. The default policy rejects unknown keys in core namespaces and preserves extension fields only in `extras` and the top-level `plugins` namespace. To support older plugins, explicitly choose `schema_policy="permissive"`. Arrays are generally replaced as a whole rather than merged element by element.

Uploading a Setup JSON fills the form with its parsed values. Any later form edits become part of the current configuration. To reproduce a run, save or download the final complete configuration; the original upload alone will not include later form edits.

17.2 Top-level Groups

Group	Purpose
`performance`	FFT backend, worker count, and activation threshold
`physics`	Fidelity preset and individual effective physical switches
`simulation`	Run name, clock, Signal model, multipass setup, and recording policy
`multiphysics`	One-way or fixed-point coupling, relaxation, and Seed heat feedback
`grid`	Main physical grid and optional independent uniform optical grid
`material`	Concentration, geometry, refractive index, losses, spectroscopy, thermal properties, mechanics, and nonlinearities
`energy_levels`	Levels, transitions, cross sections, lifetimes, and nonlinear rate processes
`pump`	Pump beam, waveform, spectrum, and propagation method
`seed`	Seed energy, complex time/frequency fields, and spatial field; the system repetition rate is actually stored at `simulation.repetition_rate_Hz`
`optical_train`	Shared sequential Pump/Seed paths before the first crystal hit and after the last hit
`ase`	Six-direction or discrete-angle ASE transport, reflecting boundaries, and parasitic-threshold parameters
`schema_policy` / `plugins`	Strict core-key policy and open plugin namespace

The default is `schema_policy="strict_core_with_plugin_extras"`. Unknown fields in core default namespaces cause an error, preventing misspelled fields from being retained without effect. Put extension metadata in a module's `extras` or the top-level `plugins`. Use `schema_policy="permissive"` explicitly only when compatibility with free-form fields in older plugins is needed.

17.3 A Readable Minimal Starting Point

The example below illustrates the configuration structure. Its values do not describe a particular material, and omitted fields are filled from defaults:

```
{
  "simulation": {
    "name": "my_amplifier",
    "signal_model": "fluence",
    "multipass": {"enabled": false},
    "repetition_rate_Hz": 25000000
  },
  "grid": {
    "size_x_m": 0.004,
    "size_y_m": 0.002,
    "length_z_m": 0.009,
    "nx": 64,
    "ny": 48,
    "nz": 64
  },
  "material": {
    "name": "replace_me",
    "ion_density_m3": 7e24,
    "refractive_index": 2.2,
    "geometry": {
      "shape": "cuboid",
      "width_x_m": 0.004,
      "height_y_m": 0.002
    }
```

```
  },
  "pump": {
    "power_W": 14.4,
    "wavelength_m": 1.55e-6,
    "waist_x_m": 75e-6,
    "waist_y_m": 75e-6
  },
  "seed": {
    "pulse_energy_J": 2.8e-8,
    "wavelength_m": 2.3e-6,
    "waist_x_m": 125e-6,
    "waist_y_m": 125e-6
  },
  "energy_levels": {
    "model": "four_level",
    "pump_absorption_cross_section_m2": 1e-23,
    "signal_emission_cross_section_m2": 1e-22,
    "upper_state_lifetime_s": 5e-6
  }
}
```

Replace cross sections, total and radiative lifetimes, ion density, `n2`, and external losses with source-backed data for the actual sample. These example numbers are not recommended Cr:ZnS parameters. Nd-like rate networks and questionable anisotropic optical or mechanical fields in current material records are subject to fail-closed quarantine and cannot activate automatically.

17.4 Unit Conventions

Core JSON values use SI throughout: m, s, Hz, W, J, K, m^{-3}, m^2, and m^{-1}. Field names generally include suffixes such as `_m`, `_s`, `_Hz`, `_W`, `_J`, and `_m3`. The interface may display mm, µm, fs, nm, or MHz, but converts values back to SI when writing the configuration.

17.5 Configuration Sources and Overrides

After material import, `material.library` and `setup_provenance` track the source. If you later change ion density, cross sections, or lifetimes manually, the summary records a user override. The setup then differs from the material-library record.

---

18. Module Coupling, Conservation, and Avoiding Double Counting

18.1 Fast One-way Chain

The main `one_way` sequence is:

```
Configuration/material → 3D doping → Pump/population operating point
                                     (including applicable ASE or periodic Seed probes)
                       → Heat-source ledger → Temperature/stress/Δn
                       → Final reported Seed propagation and extraction → Results
```

The final Seed calculation uses the solved thermal field and any enabled photoelastic or end-face phase. The chosen backend determines whether the full spatial phase is applied. Seed heat coupling defaults to `off`, so the final Seed does not trigger a thermal recalculation. `one_way_postprocess` instead performs the single update described in Section 6.6.

18.2 Thermal-optical Fixed Point

`thermal_optical_fixed_point` repeats:

```
Complex Pump field → Absorption/populations/ASE → Heat and stress → Δn → New complex Pump field
```

Iteration continues until the residuals for active and passive Pump absorption, populations, temperature, `delta_n`, optional end-face sag, and-when Seed heat feedback is enabled-irreversible Seed heating power and output energy satisfy their tolerances. The final Seed uses the converged thermal field. The inner steady-population or periodic solver must also converge in the current iteration and during final closure. An unconverged inner result may initialize the next iteration, but the outer loop cannot report convergence on that basis. If Seed heat feedback is enabled, passive and strong-field Seed-loss heat is included according to the implemented feedback strategy. The interface identifies any heat contributions excluded from that fixed point.

For the final closure check, the solver first converges the inner steady-state or periodic calculation under the candidate thermal field, then calculates another thermal response and checks the residuals above. If verification passes, the final reported Seed replay uses the pre-pulse populations accepted by the inner convergence and exactly the same temperature, refractive index, photoelastic tensor, and end-face phase. The thermal field calculated for verification does not replace those inputs. Saved heat sources, temperature, and stress remain from the same accepted thermal solution. `thermal.closure_verification` separately records diagnostics for the next thermal solution; it must not be interpreted as an applied output field.

If verification fails, relaxed or accelerated iteration resumes within the remaining budget. The limit remains `maximum_iterations` budgeted iterations plus at most one final verification. Exhausting that budget without passing is explicitly reported as nonconvergence. `multiphysics.closure_checks` records all verification checks, and `coupled_map_evaluations` records the actual total number of map evaluations. The same tolerances and residual definitions apply. Population residuals normalized by total dopant density and changes between adjacent iterations do not give absolute error bounds on final outputs.

The `joint_frozen` option uses a different final verification policy; see the joint-iteration supplement at the end of this manual.

18.3 Energy and Heat Ledgers

Pump absorption is separated into active and passive absorption. In the recommended `pump_component_ledger`:

- Active absorption contributes heat according to the quantum-defect or effective heat fraction.
- Passive absorption becomes heat in full by default.
- Active Seed extraction removes stored energy and must not be counted again as absorption heat.
- Passive Seed absorption and strong-field losses contribute heat only when their corresponding switches are enabled.
- Kerr phase does not change total energy and does not generate heat.
- Escaping ASE carries energy away; passive absorption of ASE may generate heat.

Configuration validation rejects the old single fraction applied to total absorption, preventing active absorption, passive absorption, and optical-path losses from being conflated.

18.4 Quantum Defect

For effective fluorescence or laser reference wavelength `lambda_f` and Pump wavelength `lambda_p`, the ideal quantum defect is commonly written as

$$\eta_{QD} = 1 - \frac{\lambda_p}{\lambda_f}.$$

This gives the basic heat fraction per unit of active absorption. Inversion efficiency is determined separately. Pump quantum efficiency, branching, lifetimes, and upconversion affect populations separately through the rate equations and must not be multiplied into the same heat fraction again.

18.5 Rules for Avoiding Double Counting

- If Sellmeier material dispersion and manually entered GDD/TOD describe the same material, retain only one source.
- Once a material library's active absorption coefficient has been converted to a cross section, do not add it again as passive absorption.
- If a dual-domain Seed file already contains the stretched phase, do not apply the same GDD/TOD again.
- `spectral_time` and `coherent_cpa` represent different sets of degrees of freedom; do not feed the same time×frequency energy into both.
- Apply system output loss only once in the optical-element chain.
- A doping profile preserves the volume-averaged density by default when changing local density. Do not also change total density manually by the same factor.

---

19. Performance, Memory, and Apple Silicon

19.1 Main Computational Costs

Typical computational costs increase in roughly this order: ABCD/q-parameter preview, local FN, ordinary rate equations, a 3D thermal solve, dual-grid mapping, FFT at each z step, broadband FFT for each wavelength, coherent time×space iteration, a thermal-optical fixed point, and a multipoint sweep.

Roughly, spatial FFT cost grows as `Nz × Nλ × Nt × Nx × Ny × log(NxNy)`, while population-array size grows as `Nlevel × Nz × Ny × Nx`. A mode may omit some of these dimensions, but memory requirements grow rapidly when they are combined.

19.2 Recommended Order of Optimization

1. Use the live ABCD preview while aligning the optical path.
2. Use smaller Nx/Ny/Nz and a monochromatic/fluence model to rule out configuration errors.
3. Enable FFT only when diffraction, apertures, thermal phase, or non-Gaussian fields are needed.
4. Use `hybrid_abcd_fft` outside the crystal to avoid propagating a large FFT window along the entire path.
5. For broadband calculations, first reduce wavelength and time samples and perform convergence scans.
6. Start fixed-point iteration with looser tolerances to establish behavior, then tighten them.
7. Start parameter sweeps with a few points and disable expensive output recording that does not affect the target metrics.

Avoid using the maximum number of sweep processes and FFT/BLAS threads at the same time. Usually, each sweep worker should use one FFT/BLAS worker; enable internal FFT threading primarily for a single large 3D case. Reserve `save_vtk` mainly for debugging or a few final reference points. The current VTI/VTR writer uses ASCII, so it is slow and builds large text strings. Save complete interface result archives per point and load them on demand where possible. Keeping a large ZIP, the source arrays, and interface download copies in memory at once creates a substantial peak.

19.3 `performance` Settings

The following minimal JSON object shows the performance section. Merge this section into your saved configuration; it is not a complete physical setup:

```
{
  "performance": {
    "fft_backend": "auto",
    "fft_workers": 0,
    "fft_min_elements": 4096
  }
}
```

- `auto` uses SciPy if it is available and the total array element count reaches `fft_min_elements`; otherwise it uses NumPy. Explicit `scipy` also obeys this threshold and availability check.

- `fft_workers=0` maps to `workers=-1` on the SciPy path, using all available CPUs. A positive integer limits workers. The NumPy path does not use this worker setting.
- `fft_min_elements` refers to the total element count of the complete transform array, not the length of a single axis.

Installing `.[performance]` also enables SciPy sparse dual-grid mapping and thermal CG, plus two types of Numba kernel:

- The fused single-transition coherent transient-gain kernel parallelizes independent x/y pixels. Within a pixel, it preserves the order of soft temporal windows, symmetric forward/reverse half-sweeps, event limiting, and population conservation. Multitransition and ensemble cases always use the general reference path.
- The axisymmetric coherent radial kernel parallelizes independent Crank-Nicolson tridiagonal systems for different frequencies. The radial recurrence within each system remains ordered, uses complex128 coefficients, and preserves the original per-system energy normalization. Selection is controlled by `seed.coherent.axisymmetric.radial_diffraction_kernel=auto|numba|numpy`; `auto` considers the number of frequency systems and Numba availability.

Without optional performance dependencies, the core numerical paths can use their corresponding reference implementations. Fallbacks for optional Signal physics backends must still pass the state-consistency checks described earlier. A fallback selected at initialization may remain in use throughout the run, and an isolated pulse may return a runtime fallback after rollback and recomputation. A backend change midway through a state map or fixed point cannot be counted as convergence. Spectral material tensors, material-dispersion factors, thermal phase, angular-spectrum transfer functions, and radial coefficients have scoped caches with hard byte budgets. Only data that may be reused are retained across z; over-budget data are not cached. Accepting a new transient or steady thermal field or concentration environment clears propagation-local caches. Spectral material caches are additionally invalidated through a monotonic `environment_revision`. Consequently, caching does not carry old-temperature cross sections into later pulses or outer thermal-optical iterations.

`seed.coherent.local_operator.multirate_gain.enabled=true` provides another form of acceleration, but it is not an exact optimization that changes only execution strategy. Diffraction, material dispersion, Kerr, and the currently accepted thermal phase are retained at every fine substep. When permitted by the gain-exponent threshold, soft-window gain/population updates from several neighboring substeps are grouped into one representative substep. Intrapulse relaxation and dynamic free-carrier paths automatically return to gain evaluation at every substep. Pure multiphoton absorption no longer forces additional gain calls; its splitting error with grouped gain must be assessed against step refinement. Any new local operator with memory that interleaves with gain must also register a blocker before compatibility can be claimed. The run report records the actual grouping count, maximum group size, and blocking reasons, and issues a warning requiring a quantitative comparison.

`seed.coherent.transient_gain.maximum_bypassed_output_energy_fraction` controls bypass of low-energy empty tail windows. Its default of `0` disables bypass completely; the CPU Fast preset uses `1e-6` per gain step. Selection uses a conservative amplification upper bound based on current populations, temperature/concentration-dependent cross sections, the material mask, and z-step size. Both the global and each illuminated material pixel's accumulated estimated output must remain within budget. Bypass does not reduce the native frequency samples of the current input grid or change the forward/reverse population-update order of active windows. `inactive_window_local_guard_maximum_pixels` defaults to 512. If the spatial size exceeds this limit, bypass is disabled and full-window calculation is restored, rather than falling back to a weaker global criterion alone. Near the 5% adaptive depletion boundary, results are checked again without bypass. Intrapulse relaxation or unsupported multiple transitions also disable it automatically. The report gives bypassed and active window counts, maximum actual input fraction, global and local estimated output upper bounds, and coupling-related fallback reasons, and requires a quantitative comparison with budget `0`.

This bypass does not merge native coherent frequency packets into fewer "conservative gain groups." In each half-sweep, the transient soft-window algorithm immediately writes each frequency's update back to the shared populations. Grouping frequencies changes the populations and gain seen by subsequent packets. It must be designed and validated as a new physical or numerical closure, rather than presented as equivalent low-level acceleration.

19.4 Why Apple GPU Is Not the Default

The main field is usually `complex128`, and every z step also updates populations, gain, thermal phase, and diagnostics in NumPy/SciPy. Moving only FFT to Metal introduces CPU↔GPU transfers that often cancel the benefit. The program therefore prioritizes native arm64 Python and multithreaded CPU FFT, without introducing a complex and potentially slower path merely to report GPU use.

If the terminal environment is x86_64/Rosetta, a native arm64 environment will often be faster. Reactivate the environment in each new terminal. For the `.venv` created using the installation instructions:

```
cd "/path/to/AmplifierSimulation"
source .venv/bin/activate
python -m streamlit run app.py
```

This virtual environment belongs to this project. Python itself may be installed system-wide, but this environment's dependencies are not automatically available to other projects.

19.5 Convergence Studies Matter More Than One Large Grid

For results used in formal analysis, compare at least two grid or temporal resolutions. Automatic CLI convergence certification additionally requires at least three valid, distinct discretizations. If output energy, peak temperature, maximum B-integral, or focal spot still changes substantially with refinement, the coarser result cannot be treated as converged.

19.6 Priorities for Further Code Optimization and Decoupling

After profiling representative cases, prioritize caches for material spectral interpolation, compiled optical paths, angular-spectrum transfer functions, and fixed-grid sparse operators. Cache keys must include wavelength, temperature/environment revision, grid, and backend. Use Numba only for confirmed per-cell rate, small-matrix, and coherent-gain inner loops; do not compile large-array FFT work already handled efficiently by NumPy/SciPy.

`solver.run`, `config.validate_config`, and the interface's form/result functions remain long, making it easy for a new field to reach only some backends. Longer-term organization should separate result DTOs/schemas from solver/output, and material-provenance requirements from configuration, then split typed validators and orchestration by Pump, levels, optics, thermal, signal, and archive. These changes would reduce missed connections between modules and give performance work clearer boundaries.

---

20. Frequently Asked Questions and Troubleshooting

Symptom	Common cause	What to do
`No module named streamlit`	UI dependencies are not installed for the current interpreter, or the correct environment is not active	Activate the environment actually used, run `python -m pip install ".[ui]"`, and confirm with `python -m pip show streamlit`
The form does not change after JSON upload	Parsing failed, a field path is incorrect, or a material has not been applied	Read the upload message and configuration differences; for a material library, select a record and click Apply
The material name changes but energy-level cross sections do not	The library was loaded but its operating point was not applied, or user overrides remain active	Confirm doping, temperature, and polarization in the material panel, apply the record, and inspect the key parameters before and after application
Entered Seed energy differs from reported input	Values refer to different planes: source, first crystal entrance, or after preceding losses	Check the energy-ledger field names; source-plane input should equal `seed.pulse_energy_J`
Average output is numerically almost the same as one pulse's energy	J and W were confused, or repetition rate was not set in the shared system clock	Check `repetition_rate_Hz` and the units in `P=E×f`

Symptom	Common cause	What to do
Steady-state output is below expectations	`cycle_average` is used despite strong single-pulse extraction, or the fixed point did not converge	Use `pulse_map`, increase the maximum cycles, and inspect population and output residuals
FFT output is much lower than ABCD output	Optical-window clipping, apertures, interface/element losses, or FFT aliasing	Inspect sampled fraction, edge fields, and the energy ledger; enlarge the window or use hybrid propagation
Output decreases when Pump power increases	Physical thermal, spectral, ASE, or nonlinear rollover, or numerical clipping	Toggle modules individually and compare temperature, overlap, ASE, loss-related heat, and window diagnostics
Two gain blocks appear in the optical-path view	Separate beam path lengths were incorrectly treated as global crystal coordinates, or an old cache is present	The current version aligns the common crystal; refresh and check that each path has only its intended `gain_medium`
A 4×2 crystal cross section looks square in 3D	Automatic view scaling or browser perspective	Switch to true-scale/orthographic viewing; use axis coordinates and slices for numerical interpretation
The beam looks enormous at z=0	Envelope units/longitudinal compression distort the view, or the source is far from focus	Hover to read D4σ and use the coordinate-radius probe to check the actual position
`ArrowInvalid: cannot mix list and non-list`	Older code placed lists and scalars in one numerical DataFrame column	Use the current code; custom result tables should convert composite values to strings/JSON first
A JOSA run reports CFL/coupling errors	Time/z sampling violates the selected mode's conditions, or required full-3D physics is disabled	Read the JOSA audit; use recommended sampling or an explicitly reduced model
A thermal-optical fixed point changes from 4.2 W to 2.2 W	Thermal lensing reduces overlap, the Pump FFT window clips, or absorption actually changes at the fixed point	Inspect per-iteration absorption, Δn, sampled fraction, and output residuals; do not compare only the final W value
Temperature/Δn is zero everywhere	Thermal physics is disabled, the heat source is zero, or the refractive-index model is off	Check the thermal switch, absorption ledger, `dn/dT`, and effective model
ASE intensity grows without bound	Round-trip gain exceeds threshold, reflectivity is too high, or boundary iteration fails to converge	Check reflectivities/cross sections and the parasitic threshold; do not merely raise the intensity ceiling
A calculation is very slow	Excessive space×wavelength×time sampling, pulse-map cycles, fixed-point iterations, or sweep size	Follow Section 19.2 to reduce complexity in stages and identify the dominant cost

20.1 Shortest Route for Investigating Numerical Problems

1. Inspect the merged configuration with `validate`.
2. Disable ASE, the thermal fixed point, strong-field effects, and coherent mode; run `fluence + one_way`.
3. Check the source-energy, Pump-absorption, and system-output ledgers.
4. Compare entrance power and radius for ABCD and FFT.
5. Enable one physical module at a time and save the differences.
6. Finally, check convergence with grid refinement, temporal sampling, and fixed-point tolerances.

---

21. Choosing a Model for the Research Question

21.1 Adjusting Only the Air Path and Focal Spot

- Disable “Insert gain crystal” in the live preview.
- Use separate q/ABCD paths for Pump and Seed.
- Insert their respective lenses and use the coordinate probe to read x/y D4σ radii.
- Once the measured air focus is reproduced, reinsert the crystal and inspect the change introduced by refractive-index interfaces.

21.2 Fast Single-pass or Multipass Energy Estimates

- Set `signal_model=fluence`.
- Use population-derived gain with `energy_levels.gain_override_m_inv=null`, or explicitly enter a justified small-signal gain.
- Select `one_way`.
- Initially disable thermal and coherent features.
- This is suitable for scanning geometry, pass count, and small-signal gain.

21.3 CW or Long-pulse, High-repetition-rate Amplification

- Set Pump to CW.
- Set Seed pulse energy and repetition rate.
- For weak extraction, `cycle_average` may be used.
- For strong extraction, use `pulse_map`.
- Check the average-energy ledger before adding the thermal field.

21.4 Depletion of the Front and Tail of a Short Pulse

- Use `spectral_time` if coherent compression is not required.
- Use `josa_transport` when the published photon-transport discretization is needed.
- Selecting `retarded_pc` makes temporal bins relatively independent of Nz.
- Use `full_3d_pc` when the prescribed full three-dimensional coupling is required, accepting the higher cost.

21.5 Gain Narrowing and Multicolor Response

- Upload or define the Seed spectrum.
- Supply the material’s Signal `sigma_abs(lambda)` and `sigma_em(lambda)`.
- Use `spectral_time` to inspect the output after wavelengths interact through shared populations.
- If spatial chromatic effects are under study, also enable `seed.spatio_spectral`.

21.6 CPA, Stretching, Compression, and SPM

- Set `signal_model=coherent_cpa`.
- Define spectral intensity and the most reliable spectral phase available.
- Track source, material, stretcher, and compressor dispersion separately to prevent duplication.
- Enable transient gain, Kerr, and strong-field losses progressively.
- Inspect compressed pulse duration, peak power, B-integral, and energy loss.

21.7 Thermal Lensing and Power Rollover

- Use reliable thermal conductivity, boundary heat-transfer data, and `dn/dT` for the material.
- Prefer `pump_component_ledger`.
- Start with one-way coupling, then use `thermal_optical_fixed_point`.
- Check the FFT window and fixed-point convergence to avoid confusing clipping with thermal rollover.
- Enable temperature-dependent cross sections/lifetimes, ASE/TPL, and Seed-loss heat when supporting data are available.

21.8 ASE and Parasitic-oscillation Risk

- Supply source-backed ASE wavelength weights, radiative lifetime, and effective reflectivity for each surface.
- Use `six_ray` for initial screening.
- Then increase the direction count with `angular_discrete_ordinates` and check convergence of the dominant boundary eigenvalue.
- Near threshold, or when curved-surface TIR is important, use boundary-fitted angular transport/coherent cavity modes and experimental verification. Numerical screening does not replace safety design.

---

22. Output Files and Field Definitions

A command-line run or an interface download produces the following main result files.

22.0 Complete Interface Result Archives: `*.ampres.zip`

Use “Download Complete Result Archive” to save a ZIP that can be reloaded in the separate Results Analysis page. It contains:

- A manifest for one run or an entire one-dimensional/two-dimensional parameter sweep.
- The baseline configuration from before the sweep and a separate complete configuration for each point after its sweep parameters were applied.
- Each completed point’s `summary.json`, full NumPy field arrays, stress arrays, and optional optical planes with their coordinates.
- Solver warnings, Python warnings, validation failures/warnings, and actual/requested Seed backends.
- The configuration at the time of each execution failure, plus exception type, message, and full traceback. Failed points remain in the sweep record without field arrays.

After a background single run completes, its complete result archive is automatically saved to disk and its actual path is displayed. This persistent file is not deleted when the page refreshes. If it exists, the download streams a direct copy rather than recompressing it. A failure to save automatically is displayed as an error; while results remain in the session, a temporary generated download is also offered.

Single-run archives are saved by default in `exports/` inside the extracted project folder. The environment variable `AMPLIFIER_SIM_UI_AUTOSAVE_DIRECTORY` can override that directory. Browser downloads are separate copies saved to the browser’s selected download location. CLI `run -o ...` instead writes the individual result files to the specified output directory. A sweep’s persisted point files are temporary working data; download its complete or partial `.ampres.zip` before discarding the sweep.

To reopen saved results, choose **Result Analysis**, upload the `.ampres.zip` through the result-package uploader, and use **Select Case** to choose a point. Loading restores the saved views without rerunning the solver or replacing the current simulation form. Review the archive’s validity and warnings before inspecting individual points. This upload is for result archives; use the simulation page’s Configuration File controls to load a Setup JSON for a new calculation.

Older archives may contain model names or diagnostic labels that this release no longer recognizes. Loading their arrays does not establish that every plot has retained its original interpretation. Use the version that produced the archive, or migrate and check a separate copy before comparing plots or reusing its configuration. Keep the original archive unchanged.

Temporary single-point and sweep archives are **generated only when clicked and streamed over the local loopback connection**. Displaying the link registers a short-lived capability. The stream from `127.0.0.1` starts only when the browser follows the link. A temporary single-point archive uses bounded disk staging. A sweep loads one persisted completed point at a time instead of keeping the whole ZIP in interface memory. Temporary files are cleaned up when download completes, fails, or is canceled. Automatically saved formal archives are not deleted as temporary staging. Links expire after roughly 30 minutes by default; refreshing the results page registers a new one. This is an entry point for a browser on the local computer, not a remote file server.

If a background sweep is incomplete or stopped, the download contains only points durably completed at that moment. Its manifest records `partial=true`, `completed_points`, `requested_points`, and the job state at download time. It neither waits for nor fabricates unfinished points. After all completed points of a canceled/failed job have been successfully archived, the background cache may be cleaned according to its retention policy. Partial downloads of a running or paused job do not remove retention requirements. If interface summaries remain but the links to complete `SimulationResult` objects for successful points have been lost, the program refuses to generate a package named “complete.” Instead, it offers an archive whose filename includes `summary_only` and whose metadata states `full_field_data_included=false`. That archive preserves metrics, configurations, validity, and warnings for each point, while explicitly excluding 3D fields, optical planes, and complete per-point results for replots.

After loading, inspect the overall trends, 2D heatmaps, per-point tables, and errors/warnings center, then select any operating point. Completed points reconnect saved results and their original configurations to the result view. Consequently, cold and post-run 3D optical paths, spatial fields, spectra/pulses, recorded optical planes, and all diagnostics are restored from the original point, without reading subsequent edits to the current simulation form. The results page reports the restored 3D grid and number of optical planes. The 3D Optical Workbench opens its rotatable view by default. The 3D Fields and Draggable Slices view uses separate control state for each archive/point, so changing points does not reuse the previous point’s field name, orientation, or slice position.

Uploaded archives are treated as untrusted input. Fail-closed limits apply to ZIP member count, compressed/decompressed size and ratio, JSON/NPZ size, array count, dimensions, axis lengths, element count, and total bytes. Manifest counts must be actual JSON integers, not booleans or fractions. Required and optional arrays are checked for numerical dtype, shape, and finiteness. The interface can first read the index, then load full fields for the selected point. It does not need to extract all points in a large sweep at once.

The library’s default `ArchiveResourceBudget` includes: `maximum_archive_bytes=1 GiB`, 4,096 members, 4 GiB total decompression, 2 GiB per member, 64 MiB JSON, 2 GiB NPZ, 4 GiB of arrays in total, 2 GiB per array, at most 5×10^8 elements per array, 8 dimensions, and 10^7 entries on one axis. The interface’s upload entry point explicitly raises the compressed-archive limit to 3 GiB while retaining the remaining prechecks. For larger results, reduce saved fields or use a controlled local-file workflow rather than disabling the budgets.

22.1 `summary.json`

This human-readable hierarchical summary contains:

- Configuration hash, selected models, and effective physics.
- Pump source power, grid-entrance power, and active/passive/total absorbed power.
- Seed source-plane input, energy for each pass, system output energy, and average power.
- Population, inversion, gain, extraction, and absorption statistics.
- Heat-source components, maximum temperature, Δn, stress, and thermal-optical residuals.
- ASE, CPA/JOSA, strong-field, and spatial-mode diagnostics.
- Irreversible Seed heating power and coupling mode (`seed_irreversible_heat_*`); older `seed_nonlinear_heat_*` fields are read as equal-value compatibility aliases.
- The `physics_validity` model-applicability report and audit of numerical population corrections.
- A deduplicated warning list.

Archive this file first, while also saving the full three-dimensional arrays separately.

22.2 `fields.npz`

Read it using NumPy:

```
import numpy as np
d = np.load("output/example/fields.npz")
print(d.files)
temperature = d["temperature_K"]
```

Fixed fields include:

Field	Physical quantity / shape
x_m,y_m,z_m	Voxel-center coordinates
x_edges_m,y_edges_m,z_edges_m	Cell boundaries
population_m3	Final population of each energy level, [level,z,y,x]
population_after_pump_m3	Post-Pump populations before Seed extraction
inversion_after_pump_m3	Post-Pump inversion defined for the Signal transition
pump_intensity_W_m2	Three-dimensional Pump intensity
pump_absorbed_W_m3	Total Pump absorbed-power density
signal_fluence_J_m2	Seed fluence selected by the recording policy
gain_m_inv / net_gain_m_inv	Final net gain inside the crystal: remaining active gain minus passive loss
initial_active_gain_m_inv / remaining_active_gain_m_inv	Active gain before/after the Seed, excluding passive loss
passive_loss_m_inv	Passive-loss coefficient
heat_W_m3	Total heat-source power density
extracted_energy_J_m3	Active net Seed-extraction energy density; a negative value indicates net absorption
temperature_K	Temperature field
delta_n	Refractive-index change due to temperature/stress
crystal_mask	Mask of the physical crystal
dopant_density_m3	Actual three-dimensional dopant density

Model-dependent optional fields include active/passive Pump absorption, passive Seed loss, nonlinear loss, ASE intensity, spectral/temporal axes, input/output spectra, spectral phase, compressed temporal power, and independent optical-grid coordinates.

22.3 optical_planes.npz

This file is produced when record_optical_planes is enabled and the solver has recorded diagnostic planes. Keys encode the beam, element, or pass; arrays are generally two-dimensional intensity/fluence or quantities derived from the complex field. Use the plane descriptions in summary.json to interpret the arrays and check which keys are present.

When saved planes have their own coordinates, optical_plane_coordinates.npz accompanies optical_planes.npz. Its keys use <plane-name>__x_m and <plane-name>__y_m. Use these coordinates for each plane rather than assuming that all planes share the physical-grid axes in fields.npz.

22.4 fields.vti / fields.vtr

- Uniform x/y grids are written as VTI.
- Nonuniform or locally refined grids are written as VTR.

Open these files in ParaView to inspect Pump, Seed, gain, heat, temperature, delta_n, upper-level fraction, doping, von Mises stress, and optional loss/ASE fields.

22.5 slices.png

These quick-look slices are generated when Pillow is installed; a NumPy-only base installation may omit this file. Use it to identify obvious decentering, clipping, or hot spots visually. Obtain quantitative values from NPZ/VTK.

22.6 Recording Levels and Instantaneous Intensity Inside the Crystal

simulation.result_recording.level defaults to simple, preserving basic final results. medium adds scalar iteration histories. detailed additionally records physical-z-layer diagnostics for supported sampled-field backends. The level changes recording density, not the governing equations. longitudinal.z_stride controls the interval between scalar records, and m2_z_stride controls sparse M² checks. Adaptive microsteps are aggregated into physical cells rather than duplicating full fields for each microstep. physics.record_optical_planes is a separate switch.

In detailed mode, coherent CPA can enable `longitudinal.intensity_snapshots=true` (off by default); `intensity_z_stride` sets the plane interval. The entrance and final-cell exit are always recorded, with intermediate planes chosen by stride. Each plane stores two actual instantaneous-intensity maps: one at the time of peak local **integrated power**, and one at the fixed retarded-time bin corresponding to the entrance peak for that pass. The fixed bin is reset at each pass entrance and is not an absolute experimental clock for the whole system. The entrance plane is after the interface/projection, and the crystal exit plane is before the exit interface and compressor. The complete set of time slices or full complex field is not saved.

Optional arrays are `longitudinal_intensity_W_m2`, `longitudinal_intensity_area_m2`, and `longitudinal_temporal_power_W`. Snapshot metadata records axis order, coordinates, time selection, z position for each plane, and projection cosine. Axisymmetric backends retain their native radial/azimuthal grid, so arrays must not always be assumed to have shape `[z,y,x]`. Intensity is defined on the beam-normal cross section. At oblique incidence, integrate total power as `sum(I × area) × projected_area_cosine`; the projection factor must not be omitted. Enabling snapshots increases array memory and archive size; disabling them does not change propagation results.

22.7 Recommended Result Records

For every case used in formal analysis, save the final JSON, `summary.json`, `fields.npz`, program Git commit/version, material-data sources, execution environment, and convergence comparisons. Screenshots alone cannot reproduce a calculation.

A command-line result directory is written as one complete generation: summary, NPZ, optional optical planes, VTK, and quick-look files are first generated in a sibling staging directory. Managed files are replaced only after every write succeeds; failure restores the previous generation. Old sidecars such as `optical_planes.npz` or VTI/VTR that were not produced for the new generation are removed. User files outside the program's managed-file list remain untouched. The summary and three-dimensional fields therefore remain from the same result generation.

---

23. Source-module Index

Physics modules are independent of the interface. The tables below group the main files by responsibility. A configuration field should first be normalized in `config.py`, then consumed by the relevant backends. The interface must not become a second source of physical defaults.

Unless a path is shown explicitly, a Python filename in these tables is located in `src/amplifier_sim/`.

23.1 Entry Points and Configuration

File	Purpose
`app.py`	Streamlit pages, form organization, Run buttons, result views, and downloads
`src/amplifier_sim/__init__.py`	Public package entry point
`src/amplifier_sim/__main__.py`	Entry point for `python -m amplifier_sim`
`cli.py`	`run/validate/models/materialize/convergence/uncertainty/calibrate/benchmark` commands
`config.py`	Defaults, recursive merging, model resolution, and physical-validity checks
`validation.py` / `strict_validation.py`	Basic result checks and strict quantitative-validity reports
`physics_validity.py`	Propagation angles, FFT carrier checks, ignored elements, and model-applicability diagnostics
`json_contract.py`	Strict JSON reading and portable writing of finite numerical values
`constants.py`	SI physical constants
`units.py` / `errors.py`	Unit conversion and public exception types

File	Purpose
`model_explanation.py`	Readable explanations of requested models, effective backends, and reduced-model assumptions
`i18n.py`	Chinese/English display messages; does not change configuration enums or numerical results
`ui_state.py` / `ui_compat.py`	Interface state and legacy-configuration compatibility transformations independent of Streamlit

23.2 Top-level Orchestration and Results

File	Purpose
`solver.py`	Assembles Pump, populations, ASE, Seed, heat, stress, and fixed-point calculations; creates `SimulationResult`
`fixed_point.py`	Fixed-point/Anderson acceleration protected by memory budgets and conservation checks
`run_context.py`	Runtime context and progress boundaries
`output.py`	Atomic summary, NPZ, VTK, and PNG writing, with cleanup of old managed sidecars
`result_recording.py`	Simple/medium/detailed recording policies, longitudinal scalars, and optional instantaneous-intensity snapshots
`result_fields.py`	Single source of schema and extraction rules for arrays in a result
`result_archive.py`	`.ampres.zip` writing, resource-budget prechecks, indexing, and per-point loading on demand
`streaming_download.py`	Local-loopback streaming of result ZIPs generated on click, and short-lived capability management
`diagnostics.py`	Interface-independent quantities such as energy and inversion diagnostics
`sweep.py`	Single-parameter sweeps, unit conversion, per-point failure isolation, and metric extraction
`sweep_plotting.py`	Sweep-curve and heatmap data preparation independent of Streamlit
`convergence_suite.py`	One-axis-at-a-time refinement and metric-convergence reports
`profiling.py`	Stage timings and memory diagnostics
`optional_signal.py`	Optional Signal backend failure policies, rollback provenance, and state-map consistency checks preventing silent mixing of physical backends
`compute_backend.py`	Shared boundary for NumPy/SciPy FFT and optional acceleration
`ui_capacity.py`	Background-worker capacity and protection against oversubscription across tasks
`ui_solve_jobs.py` / `ui_solve_worker.py`	Background single-point solves, atomic handoff, status, and failure finalization
`ui_sweep_jobs.py` / `ui_sweep_worker.py`	Pausable background sweep processes, per-point file handoff, and archiving

23.3 Grids, Geometry, and Materials

File	Purpose
`grid.py`	Uniform/locally refined Cartesian grids, boundaries, and integration
`grid_coupling.py`	Continuous-intensity sampling between the main physical and local uniform optical grids, conservative ledger write-back, and coupled propagators
`geometry.py`	Cuboid/cylindrical crystal masks and geometry summaries
`doping.py`	Geometry-aware three-dimensional doping profiles and normalization
`material_library.py`	XML/JSON material-library parsing, 1%-doping calibration, operating-point application, and source hashes
`material_range_guard.py`	Range requirements and provenance for material snapshots and runtime curves
`material_response.py`	Data-driven feedback through `sigma(lambda,T,N)` and `tau(T,N)`
`spectral.py`	Interpolation of wavelength-dependent material cross sections and optical-element spectral responses
`dispersion.py`	Sellmeier and GD/GDD/TOD/FOD phase utilities

23.4 Beams, Optical Paths, and Preview

File	Purpose
`beams.py`	Registry of Gaussian, super-Gaussian, top-hat, ring, and other transverse beams
`optics.py`	Sequential ABCD/FFT optical paths, element transfer, gain-path splitting, and phase propagation
`optical_system.py`	Shared data boundaries between sources, optical trains, and gain media
`multipass.py`	Compiles double-pass, two-mirror, Herriott, 4f, prescribed, or migrated sequential templates into common crystal-hit/inter-pass routes
`pass_routing.py` / `directions.py`	Forward/backward multipass routing, coordinate reversal, and direction requirements
`bench_preview.py`	Fast Gaussian second-moment/q-parameter preview and coordinate-radius probes
`path_view.py`	Display coordinates aligning independent local Pump/Seed paths to the shared gain medium
`ui_visualization.py`	Plotly 2D envelopes, 3D beam volumes, slices, and result plots; does not participate in solving
`spatial_modes.py`	Moment-matched incoherent HG spatial sources from M^2 and maximum mode order
`seed_spatio_spectral.py`	Wavelength-dependent waists, M^2, longitudinal chromatic shifts, spatial chirp, and angular dispersion

23.5 Pump, Energy Levels, and Ordinary Seed Models

File	Purpose
`profiles.py`	Discrete weights for CW/pulsed Pump waveforms and analytical/file spectra
`pump_source.py`	Single conversion boundary from `waist_m2`/`waist_divergence`/`emitter_na` inputs to standard radii, M², and super-Gaussian order
`pump.py`	Prescribed-caustic, ABCD, FFT, and hybrid Pump propagation with active/passive absorption
`pump_raytrace.py`	Fixed-sample stochastic Pump rays, sequential elements, and 3D path-length deposition
`pump_heat.py`	Broadband Pump quantum-defect/Stokes heat fractions weighted by local net active absorption
`levels.py`	Two-/three-/four-/custom N-level rate equations, transitions, and nonlinear processes
`event_energy.py` / `energy_ledger.py`	Per-event energy partition and global conservation ledgers
`seed_gain_transport.py`	Local complex-field, fixed-Gaussian-mode, or parabolic-fit spatial gain closures, projection, and conservative energy ledgers
`signal.py`	Monochromatic Seed, optional FN saturation or linear exponential gain, optical paths, multipass propagation, and pass-energy records
`average_signal.py` / `axisymmetric_average.py`	Cycle-averaged Seed photon-flux propagation and applicable axisymmetric reduction
`spectral_volume.py`	Blockwise access to reduced spectral-volume fields without expanding a full wavelength×3D array
`spectral_signal.py`	Ordered time×wavelength energy bins, shared populations, and gain narrowing
`causal_multipass_signal.py`	Absolute-time CFL transport, simultaneous hits, and shared local-population updates for physical multipass templates
`multipass_cfl.py`	Shared CFL rebinning, synchronous gain, and dark-relaxation kernels for causal multipass propagation
`flight_time.py`	Geometric/group optical path, explicit group delay, and inter-pass dark-evolution timing ledgers for sequential multipass propagation
`spectral_reporting.py`	Consistent spectral/temporal result summaries across backends
`commercial_app_fast.py`	Gaussian-duct/Commercial App fast layer-by-layer amplification, quadratic thermal/Kerr closures, and one-dimensional SPM
`beam_quality.py`	Second-moment M² recomputed from output sampled complex fields, beam sizes, angular spectra, and spatial-angular covariances
`field_diagnostics.py`	Reduced-model diagnostics from parabolic fits to 3D Δn/gain fields, without applying another lens during propagation
`caustic.py`	Independent multiplane caustic scans, D4σ fitting, and window/power checks

23.6 JOSA and Coherent CPA

File	Purpose
`josa_modes.py`	JOSA fidelity levels, mandatory physical overrides, and definitions shared by interface and solver
`josa_transport.py`	Explicit, PC, and retarded-PC photon-transport discretizations
`josa_signal.py`	Connects JOSA updates to the time/frequency Seed propagator
`pulse_definition.py`	Temporal/spectral intensity files, Fourier linkage, phase files, and phase retrieval from two intensity constraints
`coherent.py`	Complex-pulse construction, phase retrieval, and CPA dispersion utilities
`coherent_gain.py`	Optional time-slice-resolved transient gain saturation for coherent broadband fields
`coherent_signal.py`	CPA adapter, 3D propagation interface, and failure isolation
`coherent_step.py` / `coherent_gain_kernel.py`	Local coherent step and optional fused kernel
`axisymmetric_coherent.py` / `axisymmetric_diffraction_kernel.py`	Axisymmetric coherent fields and the radial Crank-Nicolson kernel
`small_matrix_kernel.py`	Batched small-matrix numerical kernels
`nonlinear.py`	Multiphoton absorption, free-carrier absorption/refraction, and irreversible-loss ledgers; `solver.py` assembles cycle-averaged heat sources
`experimental_gpu.py`	Explicitly experimental GPU applicability/backend entry points; not the default path

23.7 Heat, Stress, and ASE

File	Purpose
`thermal.py`	3D steady/transient heat conduction and per-face fixed-temperature/convective/adiabatic boundaries
`local_thermal_lens.py`	Layer-by-layer local quadratic thermal-lens fits in a specified elliptical neighborhood and residual diagnostics
`stress.py`	Replaceable stress solvers, isotropic/anisotropic FEM, tensor rotation, photoelastic fields, and end-face sag
`ase.py` / `ase_angular.py`	Six-direction reduction and arbitrary-angle ASE radiative transport, population feedback, and parasitic-boundary eigenvalues
`polarization.py`	Jones-field, Stokes, and photoelastic-birefringence utilities
`uncertainty.py`	Correlated Monte Carlo, strict filtering of invalid points, experimentally weighted calibration, and parameter covariance

23.8 Benchmarks, Examples, and Documentation

File/directory	Purpose and distribution status
`src/amplifier_sim/published_benchmarks.py`	Source-traceable published benchmarks separate from the complete 3D solve; included
`tests/`	Configuration, propagation, conservation, interface-state, material, thermal, JOSA, CPA, and regression tests in the research/development checkout; not included in this distribution

File/directory	Purpose and distribution status
`examples/`	Additional runnable model examples in the research/development checkout; not included here, and not necessarily final recommended values for a particular crystal
`configs/example.json`	The operating-point configuration included in this distribution
`README.md`	English installation instructions and the release entry point
`docs/User_Manual.md`	This complete English guide to the interface, architecture, configuration, models, results, and performance
Separate validation and literature-tracking document	Published-paper benchmarks, source quality, reproduced numbers, and information gaps maintained as research material; not included in this distribution

---

24. Current Limits, Validation Checklist, and Terminology

24.1 What the Current Models Cannot Replace

- This is a paraxial scalar or reduced-vector physical framework, not a full Maxwell solver.
- Ordinary FN/`spectral_time` does not retain coherent phase between frequencies. A spatial FFT may still carry the transverse wavefront of each channel, but that alone does not permit coherent pulse compression.
- `coherent_cpa` retains the complex field, but still uses SVEA, paraxial propagation, and the currently implemented rate-equation coupling.
- `effective_m2`/HG intensity combinations are reduced models. `incoherent_hg` can propagate individual modes, but maximum-entropy weights generated from M² alone are not a measured complete mutual intensity or coherent mode-competition model.
- `isotropic_proxy` is not mechanical FEM and cannot directly support fracture predictions or precise birefringence safety conclusions.
- Both isotropic and anisotropic FEM use linear small-strain mechanics. They do not include real contact/preload, coating/heat-sink mechanics, or probabilistic fracture criteria.
- Six-direction ASE is a fast screening model. Arbitrary-angle discrete directions can calculate the dominant reflecting-boundary eigenvalue, but do not include boundary-fitted curved-surface TIR/rough scattering or coherent parasitic-cavity wave eigenmodes.
- The optical bench is a sequential paraxial path and does not treat nonsequential CAD occlusion, scattering, or surface roughness.
- ABCD/hybrid does not perform full spatial reshaping by apertures or phase screens before the gain medium. `off`/`prescribed_caustic` additionally ignores the corresponding source-to-gain elements. Inspect the actual execution report in `propagation_applicability`.
- `stochastic_raytrace` uses sequential paraxial geometric rays and finite-volume path-length estimation. It is not a curved-surface CAD, stray-light, side-pumped multiple-reflection-cavity, or coherent-diffraction solver.
- The q/second-moment projection in `commercial_app_fast` discards higher-order spatial structure. An `ax is_aligned` thermal lens also explicitly discards xy coupling, so it cannot support quantitative conclusions when axis coupling exceeds the strict threshold.
- Temperature-dependent cross sections/lifetimes use imported data only; the program does not infer unknown material properties automatically.
- The program has no `artificial local saturation` switch that discards Pump power to stabilize iteration. Pump saturation arises only from self-consistent population feedback on net absorption. Numerical convergence is controlled by tolerances, relaxation, and iteration limits.
- Unmodeled complete fluorescence angular transport, refraction at local end-face normals, contact/coating mechanics, and coolant-flow details may become important at high power.
- Numerical results do not replace experimental calibration, material-batch measurements, or laser safety assessment.

- `.ampres.zip` is a portable data format, not trusted code. Load it only with archive readers that enforce resource budgets and schema/finiteness checks. Pickle objects or arbitrary paths inside an archive must not be used as an external exchange format.

24.2 Minimum Validation Checklist for Each Formal Calculation

- ☐ JSON passes `amplifier-sim validate`.
- ☐ Material name, doping, temperature, polarization, and data sources are recorded.
- ☐ All numerical JSON values use SI units.
- ☐ Pump/Seed radii have been converted consistently between the experimental D4σ, 1/e², and FWHM definitions.
- ☐ Both optical paths hit the same gain medium, and elements act only on their assigned beam.
- ☐ `physics_validity.propagation_applicability` has no unresolved ignored/reduced-order elements.
- ☐ Energy ledgers close at the source plane, crystal entrance, and system output.
- ☐ FFT boundaries and sampled fraction show no significant clipping.
- ☐ Grid, temporal sampling, and wavelength count have undergone at least one refinement comparison.
- ☐ All periodic-state/thermal-optical fixed-point residuals and inner population/thermal-solver convergence checks pass; when carriers are enabled, the third-state check also passes.
- ☐ There are no unexplained warnings, runtime fallbacks, or automatic model reductions.
- ☐ Applicability metrics for reduced models such as Commercial App, Gaussian, and axisymmetric modes remain within thresholds and have been compared with a higher-order reference calculation.
- ☐ Heat sources do not double-count active absorption, passive absorption, or dispersion.
- ☐ Results have been compared with an analytical limit, published benchmark, or experimental point.
- ☐ The final configuration, summary, complete fields, and software version are saved.

24.3 Common Terms

Term	Meaning
D4σ	Diameter 4σ, corresponding to radius 2σ; for an ideal Gaussian, this radius equals the 1/e² intensity radius
FWHM	Full width at half-maximum intensity; temporal and spectral FWHM are related only for a specified phase/pulse shape
q parameter	Complex parameter representing Gaussian beam radius and wavefront curvature for ABCD propagation
ABCD	Propagation of ideal/equivalent Gaussian beams using 2×2 paraxial matrices, without arbitrary 2D field detail
FFT/BPM	Diffraction/phase propagation of a sampled complex transverse field; represents apertures and non-Gaussian fields but is limited by the window and sampling
FN	Frantz-Nodvik fluence-saturation formula, analytically describing short-pulse energy extraction from a gain medium with stored energy
JOSA transport	The program's backend for ordered photon transport and population updates following methods such as those of Springer et al.
Inversion	Population inversion defined for a specified laser transition, not simply the upper-level population
Fluence	Time-integrated energy per unit area, in J/m²
GDD/TOD/FOD	Second/third/fourth derivatives of spectral phase with respect to angular frequency
B-integral	Kerr nonlinear phase accumulated along propagation
ASE	Amplified spontaneous emission, which depletes inversion and may produce transverse parasitic oscillation
Fixed point	Repeated execution of coupled modules until input fields/populations/temperature and related quantities stop changing between adjacent iterations

24.4 Recommended Reading Order

- First use: Sections 2→3→6→8→9→14→20.
- Building an optical path: Sections 8.3→8.4→9.7→10→12.4→21.1.
- Steady-state calculations: Sections 6.7→15→18.
- CPA: Sections 9.2→9.5→9.6→9.11→11.9→11.10→11.11→21.6.
- Thermal rollover: Sections 11→18→19→21.7.
- Developing a new module: Sections 17→18→23→24.1.

24.5 Documentation-to-source Cross-check (2026-09-08)

The documentation audit on 8 September 2026 covered the full README and manual. It checked the implementation, defaults, CLI, interface, and output schemas against the source-audit fixes, and updated the relevant chapters. The table below links each topic to its implementation; each model still requires its own convergence and experimental checks. The audit checked the source directly rather than inferring behavior from historical README text, introductory PDFs, or published-paper reproduction results. Dated audit reports and run evidence are kept locally and are not distributed with the source.

The checks included local links and contents anchors, JSON examples, CLI argument parsing, `validate` runs for four referenced configurations, two material-embedding commands, and one source benchmark. All passed, along with **105 targeted regression checks**. The local documentation and regression records retain the results. Source hashes matched the 100 Python files audited at that time. The update changed documentation only; complete three-dimensional experiments and the full historical research test collection were not rerun. These results belong to that audit and were not rerun for this English edition.

Documentation topic	Implementation and points checked
Installation, commands, and exit codes (2, 16)	pyproject.toml, cli.py: dependencies, subcommand arguments, and summary-only saving for strict-invalid results
Default models, automatic selection, and invalid combinations (6, 17)	config.py: `DEFAULTS`, `resolve_signal_model`, and `validate_config`
Form upload and dependency restoration (4, 5)	app.py, ui_compat.py: automatic population of fields, requested/effective values, and preservation of imported values
Pump peaks, windows, and single shots (8)	profiles.py: `periodic_pump_waveform`; solver.py: `_prepare_population` and `_advance_sampled_pump`
Phase signs and actual sampling (9)	coherent.py, pulse_definition.py, coherent_signal.py
Multipass propagation and group delay (10)	multipass.py, flight_time.py: compiled routes, geometric/additional delays, and intrapulse clocks
Thermal recalculation, photoelasticity, and end-face updates (6.6, 18)	solver.py: single thermal update in `run`, photoelastic/sag reinstallation, and restoration of pre-Seed state
Material ranges, cross sections, and thermal parameters (5, 11)	material_library.py, material_response.py, config.py, ui_visualization.py
KPA, carriers, and adaptive checks (9.11, 11)	nonlinear.py, coherent_step.py, coherent_gain.py
Physical refinement and conservative mapping (12)	grid.py, grid_coupling.py, convergence_suite.py
Sweep statistics, backend grouping, and gaps (13)	sweep.py, app.py: separation of run/metric validity, live 2D raw-value options, and archived valid trends
Convergence, Student's t, and calibration covariance (13.4, 16.5)	convergence_suite.py, uncertainty.py: actual-backend comparisons, unavailable states, SVD, and variance definitions
Output fields, recording, and automatic archiving (22)	result_fields.py, result_recording.py, output.py, ui_solve_worker.py, ui_sweep_jobs.py
CPU FFT thresholds and workers (19)	compute_backend.py: total-element threshold, SciPy workers, and NumPy fallback

The relevant chapters describe the remaining limitations: the live 2D raw-value display includes invalid cases; `models` does not list the complete schema; the CLI and interface diagnostic archives save strict failures differ-

ently; step-size limits do not certify error; and external phase and incoherent higher-order response still need experimental definition.

Keep this manual aligned with the implemented code. When adding a configuration field or physics plugin, update the manual, defaults, validator, and related validation checks together. Record changes in published numerical values or source evidence in the separate research validation document. Retain dated audit evidence alongside the current instructions and configuration requirements; the audit records do not replace them.

Optional Radial Grid for Commercial App Fast

In the Commercial App Fast form, “Use Axisymmetric radial/angular grid” sets `seed.commercial_app_fast.spatial_backend` (`cartesian` / `axisymmetric`), with Cartesian as the default. When enabled, Cycle Average and subsequent pulses use radial/angular sampling inside the crystal, while Gaussian-q and the selected temporal closure still govern propagation. Set the radial points and angular samples through `seed.coherent.axisymmetric.radial_bins` and `angular_samples` in the form. Commercial App retains all angular samples, without the coherent model’s m-order truncation, and continues to use its own propagation model rather than coherent radial diffraction.

This mode requires a uniform square optical window, a crystal covering that window, scalar polarization, and `local_field` gain. Radial points range from 8 to the optical grid’s number of points along one side; angular samples range from 9 to 128. Optical paths outside the crystal, population results, and thermal fields retain Cartesian interfaces. This switch does not change thermal coupling. Initial spectral-volume data are mapped in blocks without expanding a full frequency×3D-space array. Population write-back, radial boundaries, and entrance projection have validation checks. Failure of radial q projection raises an error rather than silently switching to coherent diffraction.

Radial Interface and Binning for Spectral Time

The form can set `seed.spectral_time.spatial_backend` to `cartesian` (default) or `axisymmetric`. The latter applies to both Cycle Average and pulse-by-pulse propagation. It shares the radial count, angular count, and finite-m modes in `seed.coherent.axisymmetric`, using radial Crank-Nicolson diffraction inside the crystal. Each frequency retains its own spatial wavefront, but phase coherence between frequencies, Kerr/SPM, and coherent compression are not calculated. External optical paths remain Cartesian. The physical thermal grid and outer thermal-optical iteration are unchanged.

`temporal_bins` and `spectral_bins` default to null, retaining the original resolution; the form can set upper limits. Independent temporal/spectral inputs are rebinned conservatively without rewriting source data. Cycle Average uses one mean-power channel and the same wavelength bins; pulses deplete populations in temporal-bin order. Joint time-frequency matrix rebinning, incoherent multimode HG, nonlocal gain, and radial multipass propagation are not currently supported. The same spatial requirements apply to the radial interface: a common square optical grid, scalar polarization, and compatibility with the selected m modes.

Spectral Time applies one spatial gain step per physical z slice, with half-step diffraction before and after it. This propagation does not use the coherent model’s Kerr-adaptive substeps or independent gain-group settings. Reducing temporal/wavelength bins is a numerical approximation; assess accuracy through a separate resolution-convergence check.

Initial-energy Tail Binning

Under the Seed solver-model controls, “Time and frequency · Tail binning” contains two independent switches: `seed.tail_binning.temporal_enabled` and `spectral_enabled`. Both default to false for compatibility with older configurations. The default `energy_coverage=0.999999` means 99.9999%, with original sampling retained in the central region. `temporal_tail_points` and `spectral_tail_points` both default to 3: this is the total across both tails, not three per tail. Each side receives at least one bin; the remaining bin goes to the side with more initial tail energy, or to the lower-coordinate side in a tie. Bin energies are summed and representative coordinates are energy-weighted. A zero-energy bin uses the mean coordinate.

Spectral Time supports both independent switches, with the same rule for Cartesian and radial backends. Regions are selected from the initial temporal distribution and initial frequency-energy distribution, respectively. The central region and representative bin positions stay fixed throughout the run. When enabled, each switch

takes precedence over the older manual cap; disabling it restores that cap. Older caps remain in a collapsed panel and imported values are retained. Joint time-frequency matrices do not support this independent binning.

Merged bins retain the actual edges of the original bins. `temporal_bin_edges_s` and ascending `spectral_bin_edges_Hz` are saved in NPZ and `.ampres.zip`. Power and spectral density are calculated from these edges, representing bin-averaged power and finite-bin-averaged spectral density. Bin widths are no longer inferred again from energy-weighted representative points. For older archives without saved edges, original bin widths cannot be recovered uniquely from representative points alone.

Whole-pulse gain in radial Coherent supports the frequency switch: the center uses one gain bin per frequency, and the tails are merged according to budget. Log-amplitude interpolation and local energy closure map back to the full complex frequency field. Radial diffraction updates only that same central interval; weak-tail complex fields are retained. The complete uniform FFT grid and Kerr time grid remain unchanged. The temporal switch is stored only for use by Spectral Time. Coherent Cycle Average retains its own spectral grid and does not reuse the coherent FFT indices. Cartesian Coherent and transient gain do not currently support this frequency switch; configuration validation reports that limitation explicitly.

The threshold selects the initial region only; it gives neither an output-error bound nor a guarantee of the same energy coverage throughout propagation. Original inputs remain unchanged. These form settings do not affect existing result archives or the frozen code used by jobs already running.

Diagnostics Only at Final Output

The result-recording form provides `simulation.result_recording.diagnostics_timing`. `all_propagations` preserves the original behavior. `final_output` runs the enabled longitudinal scalars, instantaneous-intensity sections, M², optical-plane, and exit-caustic diagnostics only during the Seed propagation that generates the final result for each case. Each diagnostic's existing switch still determines whether it is calculated. For M², `m2_z_stride=1` means a calculation at every physical z slice during final propagation.

Cycle Average, outer thermal iterations, internal pulse maps, failed closure checks, and one-way Seed heat-source probes do not trigger these observers. Necessary power, population, heat-source, convergence, and numerical-validation calculations continue as usual. The solver marks final timing at its reporting entry point and passes it through dual grids, primary/fallback backends, and radial helper layers. The previous timing state is restored after completion or an exception. The code does not guess the last iteration from its index, add thermal iterations, or replace the final physical state. Every case in a sweep receives final diagnostics, rather than only the final point.

When a low-level propagator is called directly, `final_output` disables observers by default. The final-report entry point of a complete `AmplifierSimulator.run()` enables them. This setting is retained in JSON, result archives, and restored forms; older files default to the original behavior.

Joint Thermal-optical Iteration

Under “A. Multiphysics coupling,” the self-consistent thermal-optical mode offers `joint` iteration and the original `nested` iteration. Joint mode requires periodic `pulse_map`. An ordinary iteration propagates a pulse, updates the thermal field, and carries populations forward from the previous iteration. The default is one pulse per heat update; `multiphysics.joint_cycles_per_iteration` controls this packet size. After output energy, populations, and outer residuals satisfy the existing consecutive-pass requirements, the thermal field is held fixed while the original inner convergence is completed, followed by outer verification. If final verification fails, iteration resumes. Reaching the iteration limit does not establish convergence. Tolerances and consecutive-pass counts use the existing form settings.

Relaxation, the outer limit, inner/outer tolerances, and final-diagnostic switches can be configured separately. If `multiphysics.iteration_scheme` is absent, the solver uses the original nested mode so that older configurations retain their meaning. Thermal-field acceleration has separate Anderson controls. Numerical convergence of joint iteration does not replace physical-validity checks.

The release also supports `multiphysics.iteration_scheme="joint_frozen"`, which is selected in the bundled `configs/example.json`. It uses the same joint updates, then verifies the final pulse map with the thermal state held fixed. Once the required inner and outer residuals have passed consecutively, it holds the accepted thermal state unchanged and runs the specified `multiphysics.frozen_probe_cycles` pulse-map checks. All required consecutive pulse checks must pass. No new thermal map is calculated after final pulse verification. If the probe fails, alternating iteration resumes within the remaining budget. The report records `closure_policy="frozen_thermal_pulse_probe"` and `final_thermal_closure_recomputed=false`, so this must not be interpreted as the recomputed thermal closure used by `joint` or `nested`. Consult both convergence diagnostics and the separate physical-validity report when interpreting the result.